École Polytechnique Fédérale de Lausanne
DEDIS

Continuous service on intermittent servers: The case for an off-grid, solar-powered P2P network

by Ismaïl Sahbane

Master Thesis

Approved by the Examining Committee:

Prof. Dr. Bryan Ford
Thesis Advisor

Pierluca Borsò-Tan
Thesis Supervisor

Prof. Dr. Jean-Yves Le Boudec
External Expert

EPFL IC IINFCOM DEDIS
BC 210 (Bâtiment BC)
Station 14
CH-1015 Lausanne

January 9, 2026

Acknowledgments

I would like to express my gratitude to the people that helped and supported me in the realization of this project.

First, I want to thank **Pierluca Borsò-Tan** and **Prof. Bryan Ford**, for their continued and thoughtful support, their understanding and their inspiring suggestions. I also feel very grateful that they accepted to host me at DEDIS in the first place and supervise the unconventional project proposal I presented them. It has been a real pleasure and a very stimulating experience to have received expert feedback on a topic that meant a lot to me.

I also want to thank **Prof. Jean-Yves Le Boudec**, which has kindly accepted to offer his experience as the expert for my project.

Finally, I would like to thank **Shailesh Mishra** for his friendly presence and precious advice, and my girlfriend and my loved ones for their unwavering support and patience throughout the semester as well as their fresh eyes and honest feedback on my work.

Lausanne, January 9, 2026 Ismaïl Sahbane

Abstract

Driven by the present need and the future inevitability of a degrowth in fossil fuels consumption across every activity sector including computing, intermittent renewable energy sources are being mass-adopted worldwide, with solar energy leading. They exhibit high space-time variability, and existing approaches to provide continuously available compute services despite this variability either rely on extensive energy storage or assume access to the power-grid, failing to completely eliminate the reliance on fossil fuels. They are also inaccessible to remote, off-grid communities.

This thesis considers a set of geo-distributed, exclusively solar-powered off-grid servers, and proposes to take advantage of complementary patterns of solar energy across latitudes and longitudes to form highly available and compact replication sets. Secondary contributions include a purpose-fit delay-tolerant routing algorithm.

Simulation results show that offering continuous availability is achievable with small replication sets and small message passing overhead, outline the importance of geographical realities on congestion and overall performances, and suggest the need for further study focused on robustness and practicality towards deployment.

Contents

Chapter 1

Introduction

In the last decade, the adoption of intermittent renewable energy sources, especially solar energy, followed an exponential trend, driven by the global pressure to reduce CO2 emissions and falling costs. The variability of power makes it challenging to use them for a large share of the energy needs of many computing applications that rely on continuous power.

Several studies took advantage of the variability of power across geo-distributed locations, to offload compute according to renewable energy availability. Often driven by datacenter operating cost minimization, they either rely on significant energy storage, which is economically and environmentally costly, or fail to entirely remove the reliance on the power grid, which is very often partly supplied through fossil fuels. This dependence on the grid also makes these solutions inadequate for regions where the power grid is unreliable or absent, despite renewable energy, especially solar, being one of the main options in such contexts.

We propose to look at the problem of organizing geo-distributed, off-grid, fully solar-powered and hence intermittent servers into highly available replication sets, allowing each participant to offer a continuous service. This corresponds to a decentralized, local and asynchronous covering problem, for which we propose an incremental greedy solution. To allow servers to discover each other and communicate with arbitrary peers in the system, which is necessary for our solution, we propose representations for availability profiles and a simple, purpose-fit delay-tolerant routing protocol.

To evaluate them, we provide an implementation, and build a purpose-fit simulation environment, including an event-driven energy module using irradiance measurements.

The general aim is to assess the feasibility, practicality and overhead of offering continuous services on such a decentralized, intermittent infrastructure, to identify challenges and relevant trade-offs, to assess the impact of geographical realities on the systems behavior and performances, and to outline a path towards a robust, real-world implementation of such a system.

The rest of this document is organized as follows: Chapter 2 reviews the general context, the literature on energy-aware, geo-distributed computing and related questions, and motivates

our specific problem framing. We specify the model and assumptions in chapter 3, identify the main sub-problems top-down, and give a high-level explanation of each proposed solution. Chapter 4 gives a more detailed view of the algorithms and protocols, and presents the simulation environment. We evaluate the entire system in chapter 5, and discuss the limitations, insights and openings suggested by the results in chapter 6.

Chapter 2

Background

As the consequences of the global environmental emergency deepen and concerns about the ability of our societies to keep operating as usual in the near future grow, the need to reduce our impact on the environment across most economic sectors is greater than ever [7]. Energy production, including electricity is one of the main sources of CO2 emissions globally, because a large part is still generated using fossil fuels [4], referred to as *brown energy*.

Beyond the climate issue, geopolitical considerations can also be a strong motivation to stray away from brown energy [41], which became especially apparent in recent conflicts [60].

On the other hand, existing computing infrastructure and its energy and hardware consumption are growing at an unprecedented rate, often using brown energy and driving up CO2 emissions [33, 40]. This makes most of the computing services used by many people and institutions depend on the power grid and global supply chains, making them vulnerable to disruptions likely to be brought about by climate catastrophes and sudden drops, or even partial collapses of regional energy supply.

Additionally, many communities in rural parts of developing countries do not have access to reliable grid power, but still have some access to the internet [16, 24]. The reliance on the grid that most computing paradigms expect makes it impractical to host services that require continuous availability in such regions, forcing the members of those communities to rely on external companies' services, often running on datacenters powered by brown energy.

2.1 Renewables for computing

Renewable energy sources appear as one of the main alternatives to brown energy [7], and among them, solar energy has by far the greatest adoption and deployment momentum globally [17, 19]. It has several advantages, which offer partial solutions to the above issues.

- It is relatively easy and inexpensive to install, even for small scale generation, and can enable some access to electricity for the aforementioned off-grid communities [36, 46].

- Solar energy is also at least seasonally available in most inhabited regions worldwide, which can help weaken dependencies between fossil fuel producer and consumer countries, and tends to be especially productive in the regions with the most off-grid communities, some of which are already rapidly adopting it as an off-grid power source for daily usage [39].

One of its main drawbacks, which it shares with wind power, is its variability, caused by many factors including seasonal patterns, day-night cycles, and cloud cover influenced by weather and climate. Even though a growing number of datacenter installations use solar energy for a share of their energy needs, either through the grid or directly thanks to behind-the-meter on-site production, they still rely on non-renewable energy sources to a large extent [20].

A common way to deal with energy variability is energy storage, which often consists of lithium-ion batteries [20]. It is known that producing them raises significant environmental and ethical concerns. Significant CO2 emissions are caused by both material extraction and manufacturing processes [21], extraction of essential materials such as lithium are associated with water depletion and waste metal pollution [2, 21, 26], and several of them are considered non-renewable resources [44], which harms sustainability objectives. Additionally, the populations living near mines in several regions suffer from exploitation, land destruction or health issues [35, 57, 59].

To continuously operate a server or datacenter on variable renewable energy, the battery needs to be very large [20], making it unsuitable as a scalable long-term solution. A Life Cycle Assessment of a stand-alone solar panel and battery system found that in such cases, the environmental cost of the battery dominates that of the entire system [5].

Previous research proposed opportunistic scheduling schemes at the scale of a single datacenter, aiming execute tasks when renewable energy is most available, and optimizing the use of energy storage, subject to service constraints [28, 56], but the variability is too large to continuously operate a large datacenter without massive and costly energy storage, making it impossible to offer services that require continuous availability.

2.2 There is always sunshine somewhere

The variability of power existing both across space and time, generation between very distant sites can be radically different, and can even exhibit high inverse-correlation due to day-night and summer-winter dichotomies [18]. At any time of day and year, it is likely that some share of Earth's land will be in daytime and receive significant sunshine.

Therefore, a set of distributed, or even geo-distributed renewable generators could in principle generate a much more stable aggregate power, and while many traditional industrial applications relying on local or regional energy would not be able to take advantage of this, a set of geo-distributed computing resources could redistribute load between each other according to their respective power levels, making a better use of this intermittent energy source [31].

This approach has been explored in many studies, mainly focused on relatively few, large datacenters [29].

Early studies proposed to migrate jobs between a few datacenters [10, 32, 34], aiming to reduce the share of brown energy used. Subsequent studies extended the array of optimization methods considered and proposed to use micro-datacenters [37], to more explicitly model different subparts of datacenter systems [43], or to jointly optimize for energy and water costs and CO2 emissions [63]. More recent works adapted those ideas to slightly different use-cases, proposing to place VMs [64] or even to route AI inference requests [45], once again to datacenters or micro-datacenters receiving the most renewable energy.

However, all of them still rely on the power grid or on significant energy storage to guarantee continuous availability. They only aim to reduce CO2 emissions compared to an energy-blind baseline, but do not try to remove the dependency from fossil fuels altogether, failing to address the aforementioned issues. Additionally, many of them have cost and datacenter exploitation profitability as their primary motivation to incorporate energy-aware considerations, as opposed to environmental concerns. Furthermore, they are mainly aimed at very large computing resources, most often controlled by a single entity, and perform global optimization on a relatively small number of sites.

2.3 Going off-grid

Another way to frame the problem is to assume instead that the geo-distributed servers are off-grid and exclusively powered by renewable energy, with energy storage insufficient to power them continuously across days and seasons, and to find ways for them to organize such that they can provide services continuously together, a framing of the problem that received much less attention in the literature.

Various computing systems running exclusively on renewable energy have been tested, often at small and anecdotal scale, with different ways of dealing with intermittent availability. In [55], the author presents a desktop workstation exclusively powered by renewable energy, without an actual battery but with an array of supercapacitors, and it usually stops being available in the evening. In [1], the server hosting the website for the LowTechMagazine is presented. The server is off-grid, and powered by a solar panel and a comparatively large battery, but still becomes unavailable occasionally in particularly poor weather conditions. Instead of treating this as a failure, the authors embrace it, and the website's UI visually communicates energy and battery levels to users.

While not addressing the above concerns about continuous availability or battery capacity, these authors and the community in which they published [13] defend this different approach, which assumes a very low environmental impact and aims to offer good computing services, instead of assuming great computing services, and trying to reduce environmental impact.

Wireless Sensor Networks (WSN) are networks of low-power sensing devices capable of communicating to accomplish sensing tasks together, typically environmental sensing. Net-

works composed of energy-harvesting nodes have been deployed [49]. To make the best use of the available energy, several energy-aware protocols have been proposed, for example for solar-aware routing [14, 23] or energy aware task-allocation [15]. These proposals helped the WSNs to function more reliably and for a much longer duration if not indefinitely. However, WSNs aren't typically aimed at offering compute services like usual servers would, and because they communicate directly without going through the internet, they cannot be geo-distributed. Hence, they cannot explicitly take advantage of complementary energy patterns at the scale of Earth.

As of writing, very few works investigated geo-distributed, off-grid and fully renewable systems, leveraging geo-distribution to offer a continuously running service.

In SEEDS [11], authors propose to use geo-distributed renewable-powered embedded nodes to cache the contents of a web-server. Nodes join and leave a CHORD [53] peer-to-peer overlay depending on their availability, and each request executed by a SEEDS node instead of the actual web-server can potentially prevent the latter to consume brown energy to handle it. Their study remains ultimately focused on reducing brown energy instead of eliminating it entirely, and the services to be replicated are not proposed by the nodes themselves, but by larger, centralized servers.

Finally, the Solar Protocol [51] proposes a simplistic decentralized system of solar and battery powered nodes hosting together a static website, routing requests to the node harvesting the highest instantaneous power.

This system was actually implemented and has been running for several years, on servers set up by trusted volunteers in several countries. While being the only work we found that proposes hosting a service entirely on solar-powered nodes with the goal of continuous availability, it is mainly of an artistic nature, and does not provide an in-depth technical proposal, implementation and simulation. All nodes in the system replicate a single service, the website, and they coordinate such that at any point in time, only a single node is actively serving it.

This means that if many nodes joined the system, most of them would be unused most of the time, even when they receive a lot of energy. This also means the system does not try to detect and take advantage of complementarity patterns between the availability of servers, which could help to make a better use of the available computing resources and energy.

2.4 Delay tolerant networks

Delay Tolerant Networks are a class of networks where the edges between nodes are not always available. They can be described as a graph $G = (V, E)$, where edges $e \in E$ have time-varying availability: $e(t) \in \{0, 1\}$ for the simple case of boolean availability. The main challenge with such networks is that end-to-end connectivity is not guaranteed, i.e. at time t, for nodes v, u, there may be no path between them where all edges are available at t. This causes usual internet routing functions (e.g. IP) to not work as intended anymore, as they rely on end-to-end connectivity [8].

For this reason, a different routing paradigm needs to be adopted, where relays store messages for some duration before forwarding them, referred to as *store and forward*. The problem of finding message routes that only go through available edges at appropriate times, and arrives to the destination as early as possible and optionally using the least hops (relays) as possible is the DTN routing problem, and it requires significantly different solutions than classical IP routing.

A large literature exists on this problem. The most frequent application considers the case of satellites orbiting Earth, having only limited contact opportunities when they pass close enough to each other [3]. Algorithms in this context assume that exact contact windows are known in advance, and routing paths are calculated once centrally. Hence, they achieve very efficient and often optimal routes, and are widely deployed for satellite communication. However, these assumptions make them unsuitable to a setup where contact opportunities are uncertain, and routing decisions are local and decentralized.

Protocols such as Spray and Wait [52] and PRoPHET [30] tolerate stochastic contacts and make local-only decisions. They can be seen as a controlled epidemic protocol (i.e., sending many copies of a message to different nodes), but where message duplications are either bounded, or made only if they increase the probability of successful delivery. They offer good performances in opportunistic networks, for example when mobile nodes (e.g., a person or vehicle) evolve in a stochastic way and contact opportunities depend on distance and movement patterns. These approaches also achieve much lower overhead than would a naive pure-epidemic protocol, but there is still a significant amount of message duplication compared to forwarding approaches, where messages are sent only along a single route. This duplication appears necessary for the highly stochastic and challenging environments they're designed for.

2.5 Covering problems

Set Cover is a classic combinatorial problem. Given a collection of subsets $\mathbf{S} = S_1, \ldots, S_n$ of universe $\mathbf{U}$ ($S_i \subset \mathbf{U} \forall i$), a sub-collection $\mathbf{C} \subset \mathbf{S}$ that covers $\mathbf{U}$ must be found with the smallest cardinality [25]. That is, $\bigcup_{S \in \mathbf{C}} S = \mathbf{U}$, and $|\mathbf{C}|$ is minimized.

Set Cover is NP-hard [25], but admits an $O(log(n))$ approximation algorithm, which is no other than the *greedy* algorithm, and it is the best possible polynomial-time algorithm (unless $P = NP$) [50]. It simply consists of iteratively choosing the set containing the most uncovered elements so far.

This problem can be generalized, by considering multiple instances that share the same collection $\mathbf{S}$ and each want to cover their own universe, with a constraint on the number of times a subset can be chosen. It is referred to as the capacitated covering problem, and is naturally NP-hard as well [9]. If the constraints are uniform, the $O(log(n))$ approximation ratio still holds [6].

The above approximation result applies to the case of a single, centralized solver with perfect information of the problem input. It is reasonable to assume that in a setup where each covering

instance makes asynchronous decisions using only local information, worst-case performance guarantees would be much worse than the theoretical results suggest. However, many NP-hard combinatorial problems tend to admit approximation algorithms that, while having very poor worst case guarantees, often yield good performances in empirical scenarios, in the absence of degenerate or adversarial inputs [38, 48]. This makes them suitable to engineering-focused, very practical application scenarios as described above.

Chapter 3

Proposal overview

In this chapter, we describe precisely the problem we considered, and provide top-down motivation and a high level overview of the different components of our proposed system.

In the following, we will use the words *server*, *peer* and *node* interchangeably.

3.1 Offering continuous services

Let us assume owners of solar-powered servers as described in chapter 2 each want to offer a different service, continuously. It might be data, compute, or a combination of both. A simple web server or even a static blog would be valid examples, as is considered in the solar protocol [51].

As the servers are running on intermittent energy, they cannot by themselves offer the service continuously, and their only solution is to replicate it on other servers, which are available when they are not. The specifics of the service are out of scope, but we assume that the cost of replicating it on another peer is not negligible, and hence that servers wish to minimize the number of services they are replicating, or at least keep it at an acceptable value.

Assuming there are many such geo-distributed servers, with geographical locations and hence availability patterns diverse enough as to ensure a significant portion of them is available at any time of year, our hypothesis is that it should in principle be possible for each peer p to find a relatively small set of peers, including itself, such that at least one is available at any time. If every peer in that set replicated p's service, continuous availability of the service would be achieved.

The high-level problem we are studying is, within a large set of intermittent peers that initially do not know most of the other peers and do not know any of their availability patterns, for them to form replication links, i.e., for each peer p, to constitute a set of *hosts* $\mathbf{h}_p$ (or *replicas*), that will replicate p's service. Figure 3.1 shows examples of such replication links.

Because every node in the system is solar-powered and intermittent, no single node can be

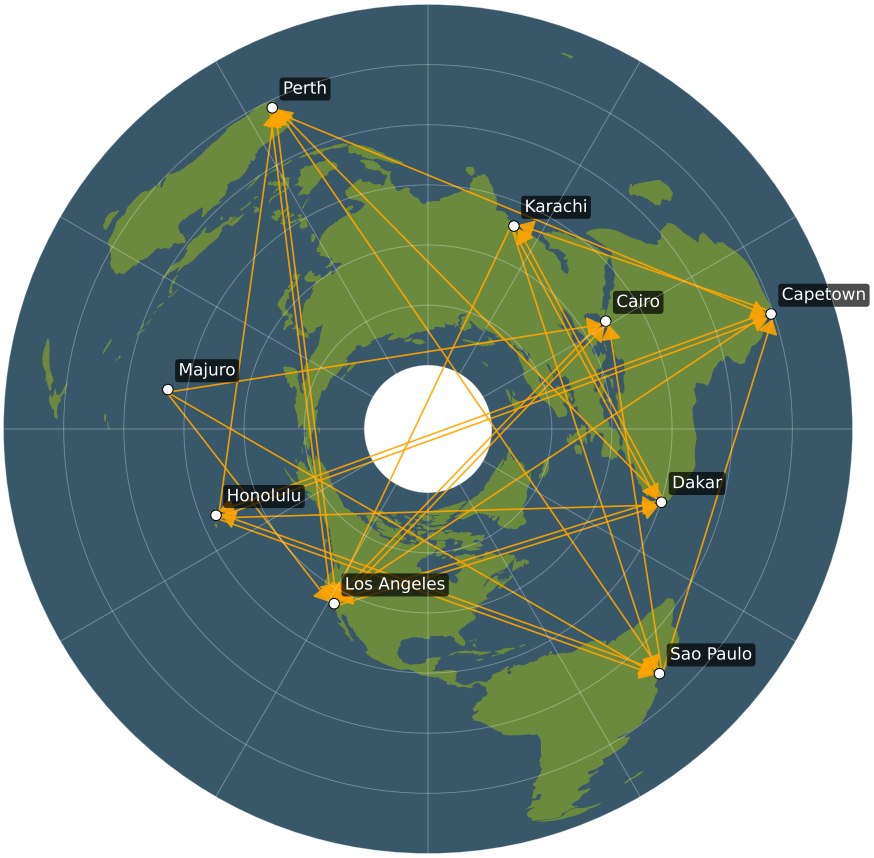


Figure 3.1: Example replication links between servers located in well-known cities around the world, using azimuthal projection. A direct yellow arrow from A to B means that B is replicating A's service

relied upon to organize the rest of the system, which motivates a decentralized, peer-to-peer (P2P) system. We emphasize that the only reason we introduce this system is to help members offer a service continuously, which they would have been able to do on their own if they were continuously available. Additionally, we are not concerned about the specific way the service will be replicated, in particular about the consistency of data in several replicas, which is a separate problem. Our task is to build and maintain the set of replicas.

As was proposed in [22] and explored by the SolarProtocol [51], we aim to design a system that adapts itself to sunlight availability in its core functions and hence *follows the sun*, a feature we designate as *heliotropic computing*.

3.2 System assumptions and scope

In this section, we present the energetic, geographic and network assumptions we made regarding system participants, and provide justification for their soundness.

3.2.1 Energy

- We assume servers are exclusively powered by an energy system with a solar panel and a small energy storage device.
- We assume the panel's peak-production is greater or equal to the server's active power consumption, so that the server can function continuously for a few hours on a sunny day.
- We assume the storage is small relative to the daily power production and consumption of servers, and in particular insufficient to power a server for an entire night.
- We assume servers know their own climate and hardware well enough or have access to enough previous harvesting power traces to create unbiased predictions of their sleep and wake-up times throughout the year.
- Finally, we assume nodes have generally low-power hardware, unable to send or process very large amount of messages.

The reasons we use energy storage are, first, to enable graceful sleep events (i.e., a server's whose energy production becomes insufficient to power it needs a short time to complete an operation, or notice its neighbors that it is sleeping now), and second, to smooth out the availability windows of servers. Indeed, the solar irradiance curve on a given day often has a roughly convex shape: it tends to rise progressively in the morning, reach a maximum and then decrease until reaching and staying at 0 for the night. With minimal storage and specific choices of sleep and wake-up triggers (detailed in 5.1.1), servers will, except in very low or degenerate energy conditions, wake up and sleep exactly once on a given day. Therefore, our sub-systems will assume a unique, non-fragmented daily availability window and optimize for it, while still being correct when that is not the case. This will simplify parts of the system's behavior, and open opportunities to make it more efficient.

Yet, in the aim of minimizing the servers environmental footprint, storage needs to remain small. In our simulations, we used storage sufficient to power the servers for only 3 hours. For reference, the SolarProtocol's nodes use enough storage to power them for about 1 day [51] and the LowTechMagazine's server uses enough to power it for 1 week [1].

3.2.2 Server location

- In our experiments, we assumed that no server has latitude higher than 60° (N or S), above which the servers may sleep for weeks. Even if our systems could support them in principle, they constitute an extreme case with complementarity patterns mainly existing at the scale of months, and their weeks-long unavailability periods would have skewed our evaluations.
- We generally assume there are nodes at locations diverse enough so that at any time of year, there is a significant fraction of them that are awake, and not close to a sleep or wake event, which guarantees overlap between availability windows.

Therefore, in our experiments, we sampled locations uniformly on Earth's land surfaces between 60°S and 60°N.

3.2.3 Network and participants

As hinted in our previous discussion on off-grid but connected rural communities, we suppose peers have access to the internet, but with potentially low bandwidth and/or high cost. Any two peers can communicate, provided they are both awake and know each other's addresses.

We designed our system with the intent of supporting a lot of peers, on the order of 10^5 or higher, even though we could not simulate that many in our experiments. We assume that upon starting, each peer p knows at least one other peer q's address, such that p and q availability windows have some overlap (even if p does not know when that overlap is), otherwise p would not be able to successfully join the network and talk with any peer.

Finally, to narrow the scope of our contribution, we adopted restrictive fault and threat models. In particular, all the nodes are not only non-byzantine, but also selfless, in that they agree replicate other nodes' services without any incentive. We also do not fully handle or simulate peer crashes unrelated to their energy availability, but discuss potential approaches in chapter 6.

3.3 Sub-Problems

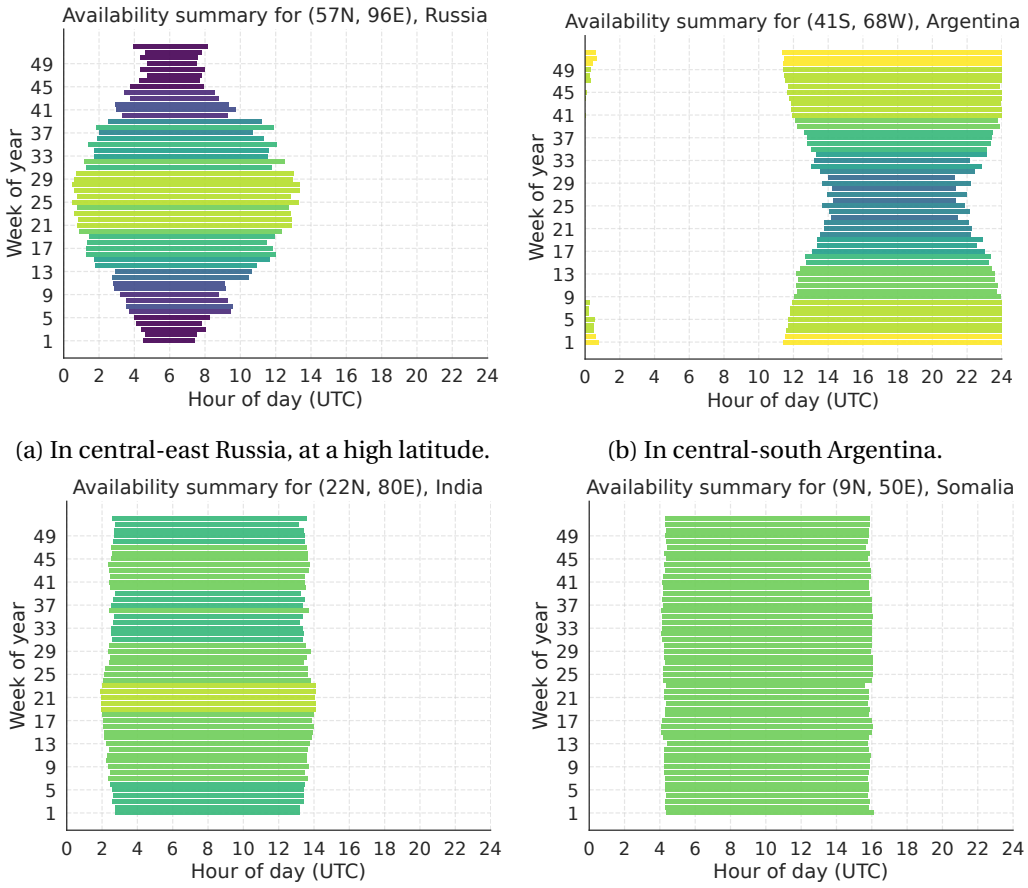
In the above, we established that our goal is for each peer to constitute a *covering* set of peers in terms of availability. We immediately identify sub-problems that need to be solved first.

3.3.1 Predicting and communicating availability

As we assume peers start by only knowing the address of a few other peers, and in the worst case, knowing nothing about their availabilities, peers must be able to exchange each other predictions of their own availability windows. There is a large set of possibilities, and in our case, as we assumed daily non-fragmented availability windows, we can represent the predicted daily window of peer p with a wake-up time and a sleep time $S_p = (\bar{uw}_p, \bar{s}_p)$, $\bar{uw}_p, \bar{s}_p \in [0, 24[$, represented as UTC hours. We call this tuple an `InstantaneousSummary`.

Obviously, due to seasonality, this summary is only representative for a few days or weeks. To represent availabilities over a full year, we simply propose to store a list of K `InstantaneousSummary` equally spaced over the year $\mathbf{S}_p = (S_p^1, \ldots, S_p^K)$, which we call a `Summary`. We can then, for any time of year, interpolate using circular interpolation on the 24-hour clock between the two closest stored `InstantaneousSummary` to estimate current wake-up and sleep times. Increasing K will increase estimates precision, at the cost of transmitting more data. In our experiments, we chose weekly observations ($K = 52$), hence a `Summary` contains $52 \cdot 2$ floating point numbers, and weighs less than 1 KB.

We compute summaries by assuming access to previous years wake and sleep times (which in our case means simulating them, see 5.1), aggregating them by week and computing their circular mean.



(a) In central-east Russia, at a high latitude.

(b) In central-south Argentina.

(c) In central India. Summer availability is affected by the monsoon season.

(d) In Somalia, near the equator. Availability is mostly invariant.

Figure 3.2: Predicted availability summaries for 4 locations. Horizontal bars length is proportional to daily availability window durations, and color visually represents their duration as well, from dark blue below 4 hours to bright yellow above 13 hours.

Figure 3.2 show `Summary` examples for chosen randomly sampled locations, displaying the wide variety of availability windows, and the complementarity they display from geographically opposite locations. In particular, in 3.2c, we observe that availability is not only driven by day length, but also by cloud cover, as South Asia's monsoon season significantly decreases availability from around weeks 25-30, even though days are the longest at this time.

3.3.2 Peer and summary discovery

We assumed previously that before joining, peers only need to know a few other peers' addresses, and do not need to know any of their `Summary`. However, to constitute efficient covering replication sets, they need to know the address and summary of a sufficient number of peers.

Since no summary is known initially, there is no other choice than to send messages to random known peers at regular intervals Δ_d, until one answers. At this point, peers exchange all the addresses and corresponding summaries they know. Once enough peers are known by some node, it stops blindly polling addresses, and only communicates when it is likely the destination is awake, to avoid wasting bandwidth and energy.

Additionally, in our system, an established peer that learns about a new peer will notice all of its *contacts*, which are types of neighbors we will define shortly, to ensure all peers eventually get informed of new arrivals.

3.3.3 Delay-tolerant routing

In many cases, nodes will want to communicate with peers which are rarely or never available at the same time as them.

Figures 3.2a and 3.2b show a typical example: The two servers seem very fit to replicate each other's services, but in expectation, they are never awake at the same time, so they cannot directly establish replication links. On the other hand, the server in 3.2d has, in expectation, significant overlap with both of them throughout the year. Hence, it can act as a relay between them. For example, it can receive messages from the first before it sleeps, store them for a few hours, then transmit them to the second when it wakes up.

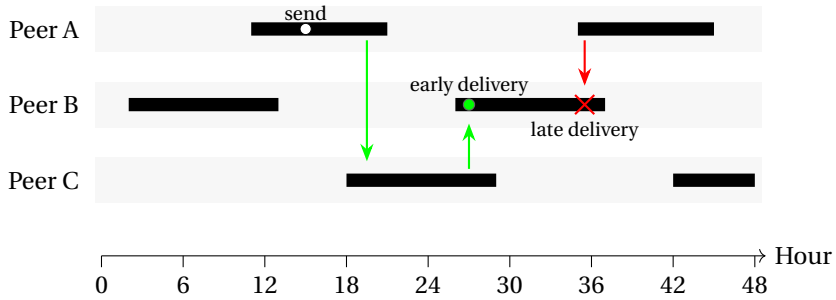


Figure 3.3: Diagram representing availability windows of three peers, A sends a message to B at 15:00, with or without passing through relay C. Arrows represent message forwarding. Passing through C enables earlier delivery.

Even when peers have significant overlap, it can be beneficial to use a relay when sending a message at certain times, so that it can arrive earlier. Figure 3.3 shows an example of such a case, where peer A wants to send a message to B when the latter is sleeping. Since they have some daily overlap, A could wait until the next day when it wakes up to deliver the message directly. However, if it relays it through C before sleeping, C can deliver it to B as soon as it wakes up, 9 hours earlier in this example.

The system we described corresponds to a specific case of Delay Tolerant Networks. In our context with decentralized and local decisions and uncertain availability windows, we can't use the optimal centralized algorithms discussed in chapter 2.

Additionally, instead of considering intermittent edges as usual DTN routing algorithms do, we only consider intermittent nodes. That is, if two nodes are awake, they can always communicate (through the internet). This is a special case of DTNs (a node being unavailable is equivalent to all of its incident edges being unavailable), and arguably a much simpler case. This fact combined with the specificity of the nodes availabilities, which are close to daily-periodic and non-fragmented, make our case significantly simpler than general DTNs.

While we could use the stochastic protocols we presented in chapter 2, we propose instead a simpler, purpose-fit approach that avoids unnecessary message duplication, while still aiming for optimal delivery times and hop counts.

Forward west The main idea of our solution consists, for a peer p about to sleep at time s_p, holding a message whose destination q is not available at s_p, to forward it to a peer r_1 which is likely to be awake for a few more hours. If a diverse set of availabilities exist in the network, there should always be awake peers that are not about to sleep, and they can carry the message for a non-zero duration into the future. Then, r_1 can do the same when it is about to sleep, and forward it to r_2, and so on.

This simple mechanism ensures that at any time, the message is held by an awake peer.

The earliest time q could receive the message is at its next wake-up time w_q. If a prediction $\bar{w_q}$ of w_q is stored along with the message, any relay r holding it around $\bar{w_q}$ can try to transmit it to q. Depending on the prediction's accuracy and in most cases, delivery times should approach the earliest possible ones. Additionally, if messages are transmitted as late as possible, and if relays are chosen to be the peers that are expected to stay awake for the longest, the number of hops used should also approach the minimum possible.

In general, the chosen relays have availabilities shifted into the future compared to the sender, and they will tend to be located towards their west. This means the messages will travel in the same direction as the sun, and this solution is an example of *heliotropic computing*.

3.3.4 Contacts

With the system presented so far, each peer p knows many other peers and their summaries and can assess for each of them how likely it is they are awake, but it still does not know any peer's status with certainty. Each time it wants to communicate directly with another peer q, it first has to poll q to see if it is awake.

To avoid having to do that every time and to simplify forwarding relay messages and other direct message passing operations, we introduce *contacts*, which can be seen as neighbors in a peer-to-peer overlay. Two peers p and q are contacts if they always know whether the other is awake or not. Maintaining this knowledge allows them to communicate directly, and in particular as soon as one wakes-up when the other was already awake. Peers become contacts if

they plan to communicate frequently in the future.

In our system, peers use contacts for two reasons:

- The gossip discovery mechanism as previously explained
- The choice of a relay, in the DTN routing protocol

In the latter case, and to minimize hop count, a peer prefers to wait until the last moment (at sleep time) to forward the messages it carries. At this point, it does not really have sufficient time anymore to poll peers at regular intervals, so maintaining a set of contacts which are typically awake a few hours after itself allows it to find an awake relay immediately.

Additionally, knowing which of its contacts is awake at this time lets it take advantage of the occasions where one of them stays awake for significantly longer than predicted or wakes-up significantly earlier, for example on sunny days in usually cloudy periods. Without contacts, the only way to know such a peer is awake is to always poll peers well outside their predicted availability, which wastes many messages

Contact set formation Each peer aims to maintain a reliable set of contacts, covering its own availability hours and several hours after its average sleep time. As long as the coverage is not satisfying, it chooses its preferred marginal set of potential contacts among known peers, and asks them to establish a contact link. To avoid blind polling, it only asks them when it is likely they are indeed awake. The set choice uses random sampling weighted by desirability (how much a potential contact improves coverage), and has many similarities with the choice of replication links.

Communication without contacts As mentioned in the presentation of our DTN routing procedure, when a relay r holds a message destined to q and time approaches the predicted wake-up time $\bar{uw}_q$, r needs to transmit it to q. If they are contacts, it is easy for r to do it as early as q actually wakes-up, but if they are not, r has no other choice than to poll q until it wakes-up. To ensure that q receives the message as early as possible most of the time, r greedily starts to poll q some time Δ_g before $\bar{uw}_q$, and after $\bar{s}_q$, anticipating early wake-up $w_q < \bar{uw}_q$ and late sleep times $s_q > \bar{s}_q$, in the hope not to miss any communication opportunity.

3.4 Forming replication sets

Finally, we use the above building blocks to introduce our replication set routine, which maintains at each peer p a covering set of hosts $\mathbf{h_p}$. Each $h \in \mathbf{h_p}$ considers p as a *guest*, and hence maintains a set of guests $\mathbf{g_p}$, containing p, of bounded size g_{max}, to prevent overload. In our system, each peer can play both the role of host and guest for several other peers, and the links are directed, i.e., if p replicates q's service, q does not necessarily replicate p's.

We observe that this problem corresponds to a decentralized, local-information and asynchronous version of capacitated set-cover, which was presented in chapter 2. Each covering instance corresponds to a peer in the system trying to cover its yearly replication window, while

the subsets available are simply the availability windows of other peers in the system. Each subset hence has capacity g_{max}

For this reason, we use a local-greedy algorithm, where each peer asks the peers filling the largest gap in its current replication set first, and they accept selflessly as long as they have remaining capacity. Our objectives are, that each peer finds a satisfying replication set, that the sets stabilize in bounded and reasonable time, and that the total number of links is minimized.

Complementarity To choose which peers to ask for replication, we need a measure of complementarity between availability profiles. Our approach assumes peers have a desired replication factor, d_r, expressing that at any time of year, p aims for at least d_r of its hosts (or itself) to be awake. Setting $d_r > 1$ corresponds to aiming for redundancy, helpful if some hosts fail or receive abnormally low energy.

For its current set $\mathbf{h_p}$, p computes, for each week and for each hour of day, the expected number of hosts awake at this time, using their `Summary`. The complementarity of an additional host candidate q w.r.t. $\mathbf{h_p}$ is the proportion of useful hours offered by q in the year. A useful hour is an hour when q is typically available, and less than d_r hosts replicating p's service are typically available. Consequently, any hour when q is available but enough hosts already are as well is considered useless, and the higher q's complementarity with $\mathbf{h_p}$, the closer it would bring p to its availability coverage objective.

The routine As with contact set formation, at regular intervals and until it has a satisfying replication set, p iteratively samples, among its known set of peers obtained through the discovery protocol, desired replication hosts using hypothetical marginal complementarity scores as sampling probabilities (we will detail the specific method in chapter 4), and sends them a request through the DTN communication layer.

Upon receiving it, they answer through the DTN communication layer as well, positively, or negatively if they already reached their maximum number of guests. Due to the nature of the network, in many cases, responses can only arrive several hours later (in the worst case, if p contacts q around w_p, and they have no availability overlap, p will only receive the answer the next day when it wakes up, close to 24h after).

To avoid asking too many peers and establishing too much redundant links, p waits a few hours before asking more peers. This results in long delays until replication sets convergence (1-3 days), and suggests a trade-off between the stability of the protocol and its convergence speed, which we will explore in chapter 5.

Chapter 4

Implementation and system details

In this chapter, we give a more precise description of the different sub-systems we implemented, and present the simulation framework we used to evaluate them.

4.1 DTN routing

Our DTN communication protocol mainly works thanks to three procedures.

Algorithm 1 Relay send (DTN Routing)

```
1: procedure RelaySend(msg, dest)
2:     if dest ∈ contacts ∧ dest.status = Awake then
3:         DirectSend(msg, dest)
4:     else
5:         pending[dest] ← pending[dest] ∪ msg
6:     end if
7: end procedure
```

First, RelaySend (algorithm 1), is called by an initial sending peer. It simply logs the message with its destination in a table for pending relay messages. Once a relay holds a pending message for $dest$, it can either try to transmit it directly if it estimates $dest$ is awake, or forward it to another relay when it is about to sleep.

For the first case, the routine shown in 2, which runs at every peer from the start, continuously finds, among the destinations for which pending messages are held, the one with the earliest opportunity to try to send its pending messages to. This is parametrized by Δ_g, which controls how many minutes before and after the usual wake-up time of a destination the routine starts and stops to try to contact it, and the retry delay Δ_r, the duration it waits between attempts. NextAvailableTime(q, Δ_g, t) considers the window $[\bar{w_q} - \Delta_g, \bar{s_q} + \Delta_g]$ and returns the smallest $t' \geq t$ in this window, modulo 24. Those parameters encompass the trade-off between transmission delay and message passing cost. In our implementation, `Relay` messages to q store $S_q(t_{send})$, so that any relay can perform this operation, even if it does not know $\mathbf{S}_q$.

Algorithm 2 DTN transmitting loop (simplified)

1: $\forall dest,\ pending[dest] \leftarrow \emptyset$
2: $\forall dest,\ lastTry[dest] \leftarrow -\infty$
3: **loop**
4: $(dest_{\min}, t_{\min}) \leftarrow \arg\min_{dest \in pending \setminus contacts} \max(t_{dest}, lastTry[dest] + \Delta_r)$
5: **where**
6: $t_{dest} = \text{NEXTAVAILABLETIME}(dest, \Delta_g, \text{NOW})$
7: SLEEPUNTIL($t_{\min}$)
8: DIRECTSEND(PING, $dest_{\min}$)
9: $lastTry[dest_{\min}] \leftarrow \text{NOW}$
10: **end loop**

A PING message is sent to that destination, to avoid sending potentially large messages without knowing if the peer is actually awake. Upon receiving a PING answer, the sender actually transmits $msgs = pending[dest]$ to the destination and flushes its pending queue.

Algorithm 3 Forward pending messages

1: **if** $\forall c \in contacts, c.status \neq$ `Awake` **then return**
2: **end if**
3: $r_{next} \leftarrow \arg\max_{c \in contacts | c.status = \texttt{Awake}} \bar{w}_c$
4: $msgs \leftarrow \emptyset$
5: **for** $(dest, destMsgs) \in pending$ **do**
6: $msgs \leftarrow msgs \cup destMsgs$
7: **end for**
8: DIRECTSEND($msgs, r_{next}$)

In the second case, upon sleeping, the routine 3 is called. It chooses, among contacts known to be awake, the one that is expected to sleep the latest, and forwards every single pending message to that contact, which becomes responsible for those messages. If no contact is awake at this time (which can happen, soon after joining, or if peers in the network have poor geographical distribution), the peer simply keeps the messages, and will retry to transmit them once it wakes up.

4.2 Replication links

Procedure 4 presents a simplified version of our replication link formation and maintenance routine, which runs continuously at each peer.

COMPUTEAVAILABILITIES uses the windows from hosts summaries to make an hourly-discretized representation of the expected amount of available hosts throughout the year. Similarly to what was discussed above, the stochasticity of availability windows may cause peers to wake-up or sleep significantly later, respectively earlier than indicated in their summary. To guard against such cases, we use a margin Δ_m to reduce the availability windows of hosts and candidates. Δ_m works in an opposite way to Δ_g, by being pessimistic about windows instead of optimistic.

Algorithm 4 Replication routine (simplified)

```
1: hosts ← ∅
2: loop
3:     avail ← ComputeAvailabilities(hosts)
4:     hosts ← GarbageCollect(hosts, avail)
5:     if not ReplicationSatisfied(avail) then
6:         desiredHosts ← GreedyHosts(peers, hosts, avail)
7:         for all h ∈ desiredHosts do
8:             RelaySend(HostRequest, h)
9:         end for
10:    end if
11:    Wait(Δ_h)
12: end loop
```

Garbage collection consists of iteratively finding the most *useless* host and removing it, until no more peer is considered useless. We define the *uselessness* with respect to p of a host h with `Summary` $\mathbf{S}_h$ as $\min_{t \in A(\mathbf{S}_h)} \max(|\{h' \in \mathbf{h}_p | t \in A(\mathbf{S}'_h)\}| - d_r, 0)$, where $A(\mathbf{S}_h)$ is the set of year times when h is predicted to be available by $\mathbf{S}_h$. In other words, h has uselessness u if at any time when h is available, at least $d_r + u$ hosts are available. A uselessness of 0 means removing h from hosts would make the replication objective unsatisfied. To remove it from hosts, p RelaySend it an UnHosting message.

Algorithm 5 Greedy host selection (simplified)

```
1: procedure GreedyHosts(peers, hosts, avail)
2:     candidates ← FilterCandidates(peers)
3:     hypAvail ← avail
4:     selected ← ∅
5:     loop
6:         scores ← MarginalComplementarity(hypAvail, candidates)
7:         if scores = ∅ then
8:             break
9:         end if
10:        h ← SampleSoftmax(scores, T)
11:        selected ← selected ∪ {h}
12:        hypAvail ← hypAvail + h.avail
13:        candidates ← candidates \ {h}
14:    end loop
15:    return selected
16: end procedure
```

Procedure 5 describes the incremental greedy procedure p uses to choose which peers to send HostingRequest messages to. Candidates correspond to p's known peers, without existing hosts and peers who refused recently. As explained, sending a single request and waiting for the answer would be too slow, as DTN round-trip times can take up to 24 hours. Sampling multiple peers independently would not be adequate either, as p risks asking multiple peers to fill the same gap in its aggregate availability windows. In case they all accept, it would end up with too many hosts.

Instead, p builds a *hypothetical* set of hosts and corresponding availabilities *avail*, iteratively samples candidates using a modified version of the *softmax* of their scores as probabilities, and update the hypothetical availabilities. Each time, it computes new scores as if all previous sampled candidates accepted its requests and were part of $\mathbf{h}_p$, combining its actual hosts' availabilities with the hypothetical ones.

MARGINALCOMPLEMENTARITY returns, for each candidate c, the complementarity improvement p would obtain from adding c to hypothetical hosts, only if this improvement is strictly positive. When no candidate can improve its host set, p stops sampling candidates.

In the replication routine, p wait for Δ_h and perform this operation again, adapting its requests to the answers it received in the meantime, in a closed loop.

Simple handlers complete the protocol: Upon receiving a new HOSTINGREQUEST, host h send a HOSTINGACCEPT message and add the requesting peer to $\mathbf{g}_h$ if $|\mathbf{g}_h| < g_{max}$, otherwise it sends a HOSTINGREFUSE message. Upon receiving a HOSTINGREFUSE, p simply logs that the sender recently refused, and upon receiving a HOSTINGACCEPT, it adds the sender to its *hosts*. This ensures that p only considers the replication link as established after both parties accepted it.

It is also possible for a peer, for example when it is stopping, to send an UNGUESTING to one of its guests, upon which the receiver will remove the peer from its hosts, and potentially notify its own replication routine that the replication set is not satisfying anymore.

4.3 Contact maintenance

To form and maintain contacts, we use a routine with many similarities to the replication routine, presented in 4 and 4.2, so we do not describe it in as much detail.

Instead of considering yearly availabilities, only `InstantaneousSummary` are used (average wake-up and sleep time), and instead of aiming for good full year coverage, peers only need contacts whose availabilities cover, for the current season, a few hours before and after their expected sleep time $\bar{s}$.

Peer p uses the same hypothetical desired contact set building method to find contacts such that d_c are predicted available before and after $\bar{s_p}$, but instead of communicating through the DTN layer (RELAYSEND), it establishes contacts directly (as contacts are meant for direct communication). Therefore, p must wait until the first desired contact is expected to wake up, at which point it tries to establish contact with it by sending STATUSNOTICE messages.

Garbage collection works a bit differently since contact links are bidirectional. When a peer detects a contact is *useless*, it cannot remove it unilaterally, since the contact might still desire the link. Instead, it flags the contact as *unwanted*. When exchanging STATUSNOTICE messages, this flag will be sent along, and when 2 peers realize they both do not need their contact link, they will remove each other.

It can happen that 2 peers previously had a significant overlap window, but they lost it due to seasonality (e.g., 2 distant peers in the Northern Hemisphere may establish a contact link in July, but will not be able to communicate at all from October). Such dead links are detected and removed as well.

Maintaining contact statuses Once a contact link is established, 2 peers will add each other to their contact set and mark each other as `Awake`. When peer p sleeps, it notifies all of its contact $\mathbf{c}_p$ with a StatusNotice, and all the awake ones receive the message and mark p as `Asleep`. Finally, when p wakes-up, it marks all of its contacts as `Asleep`, and notifies all of them (also with a StatusNotice) that it just woke-up. The ones which are awake respond with a StatusNotice that they are awake as well, and p marks them as `Awake`.

Assuming reliable communication between awake peers, this simple graceful join-leave mechanism ensures that when p is awake, it always knows whether each of its contacts is awake or not, at the cost of sending and receiving $2 \cdot |\mathbf{c}_p|$ status messages per day.

Chapter 5

Evaluation

This chapter presents our simulation and evaluation environment, the experiments we conducted, and the results we obtained. After quantifying availability prediction errors, we evaluate the discovery routine and DTN communication, evaluate the replication procedure when it relies on these components, and study the influence of geographical features on the systems behavior.

5.1 Simulation Environment

We implemented the described system in go, inside a peer object using abstract `energy`, `clock` and `transport` interfaces, so that we could swap real modules reporting measured IO with modules simulating energy, time and packet transport respectively. This was necessary in our unique setup: we needed to simulate hundreds of peers on the same machine, harvesting sunlight energy and running for many months. The code and instructions to reproduce it are available at `https://github.com/dedis/student_25_solards`.

5.1.1 Energy module

Due to the impracticality of setting up hundreds of solar-powered servers around the world, we simulated the energy hardware of each peer. The aim of this work was not to study the specifics of solar energy harvesting, but only to work with energy levels whose scale and diversity would be of realistic magnitude. Hence, we adopted a simple model assuming linear charging of the energy storage component, and modelled energy consumption as constant $P_{cons}[W]$ when the server is active (available), and $0[W]$ when it is sleeping or out of charge.

We define the charge at time t as $C(t)[Wh]$, and its capacity as $C_{max}[Wh]$, so that $0 \leq C(t) \leq C_{max}$. If the harvested power is $H(t)[W]$, we define the net power as $P(t) = \mathbb{1}\{C(t) \neq C_{max}\} \cdot H(t) - \mathbb{1}\{active\} \cdot P_{cons}$. Because $\frac{dC(t)}{dt} = P(t)$, we obtain that over a time period $[t_0, t_1]$,

charge evolves according to

$$C(t_1) - C(t_0) = \int_{t_0}^{t_1} P(t)\,dt$$

The data we used came as hourly time series, in Watts. We used the linear interpolation of the hourly measurements, that way $P(t)$ is piece-wise linear function, and therefore $C(t)$ is a piece-wise quadratic function, which we can represent, for t in an interval $[t_0, t_1]$ in which the consumption doesn't change (assume P_{cons}), and the battery isn't full, as

$$C(t) = (H(t_1) - H(t_0)) \cdot (t - t_0)^2 + (H(t_0) - P_{cons}) \cdot (t - t_0) + C(t_0)$$

where the 3 coefficients are constants. They change each time one of the following events occur: The server sleeps or wake-up (i.e. consumption changes), a new irradiance measure is reached ($H(t)$ slope changes), charge becomes empty or full, or a full battery starts discharging again. The latter happens when $H(t)$ becomes smaller than P_{cons}.

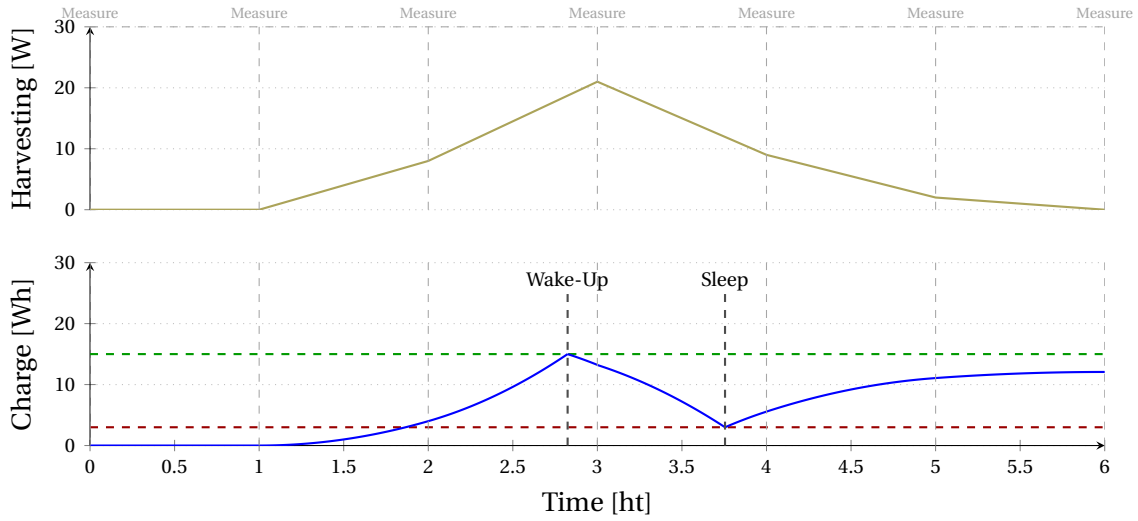


Figure 5.1: Example energy harvesting and charge evolution over 6 hours. The top graph shows the linearly interpolated harvesting power, with a new measure every hour. The bottom shows the resulting charge values if the battery started at 0. Red and Green dashed lines are sleep and wake-up thresholds, at 10% and 50%. Blue line is the charge level.

Representing the charge like this allows us to simulate it in a purely event-based way, i.e., as long as the coefficients do not change, the energy module has nothing to do, and can simply sleep until the next predicted change of coefficients occur.

This worked well with the time fast-forwarding environment (presented below), and let us avoid a loop waking-up at regular short intervals to update energy state. Predicting the next event involves finding the earliest intersection between the current quadratic charge curve, and one of the event-triggering lines.

Our servers have two charge thresholds, $\tau_{sleep}[Wh]$ and $\tau_{wake}[Wh]$, with $\tau_{sleep} < \tau_{wake}$, which trigger the corresponding event when crossed. Setting $\tau_{sleep} = \tau_{wake}$ creates an oscillation problem, where at the start and end of days, peers successively wake up and sleep many times

rapidly, and setting a sufficient buffer between them suppresses this problem. In our simulations, we used $\tau_{sleep} = 0.1 \cdot C_{max}$, and $\tau_{wake} = 0.5 \cdot C_{max}$, which smooths out the servers active windows, and additionally ensures that peers only wake-up when they are certain they have enough charge to function for a significant duration.

Figure 5.1 shows an example of charge behavior in function of harvested energy. The events, shown as vertical dashed lines, which are 7 measurements at each hour, and one wake-up and one sleep events, are the only moments where the module needs to update the state.

Energy harvesting data We used the service `renewables.ninja` [42, 47], which can generate, for any coordinates on earth, solar panel tilt and year, using historical weather data and physical modelling, hourly power time series for a hypothetical solar panel at this location for the entire year. We automatically fetched data for 800 such locations, uniformly sampled on land surfaces between latitudes $60°S$ to $60°N$, for the years 2019 to 2024. We used years 2019 to 2023 to compute the `Summary` predictions, and 2024 in actual system simulations.

5.1.2 Advancing time

In our protocols, servers typically wait for several hours, and some of our features can only be assessed when seasons progress over several months. Simulating the peers, which sleep most of the time, in real time, would have been impossible. Thankfully, `go` released a package called `synctest`, which lets tests functions create a *bubble*, and in every goroutine spawned within this bubble, calls to `time.Sleep` or `time.After` are fast-forwarded. That is, when every goroutine is waiting for time to pass, the scheduler skips to the one which finishes first.

This is very useful, as it lets us run the same code we would have used in deployment, with normal sleep operations, inside such a bubble, and simulate it running for several months in only a few seconds in wall-clock time.

5.1.3 Transport layer

Using synctest is unfortunately incompatible with real socket operations, as they are waiting for OS-level events which synctest cannot detect, and would cause peers to become deadlocked. Instead, we used an emulated transport layer using go's channels.

5.2 Summary accuracy

Before evaluating the actual system, we first want to understand the variability of the wake-up and sleep times we simulated, and more specifically, the distribution of the prediction errors with respect to computed `Summary`. As the predictions are averages over previous years, the error has very low bias, but can exhibit very high worst case variance, depending on the climate and latitude. Figure 5.2 shows, for each location, the median and the 95th percentile of the absolute error.

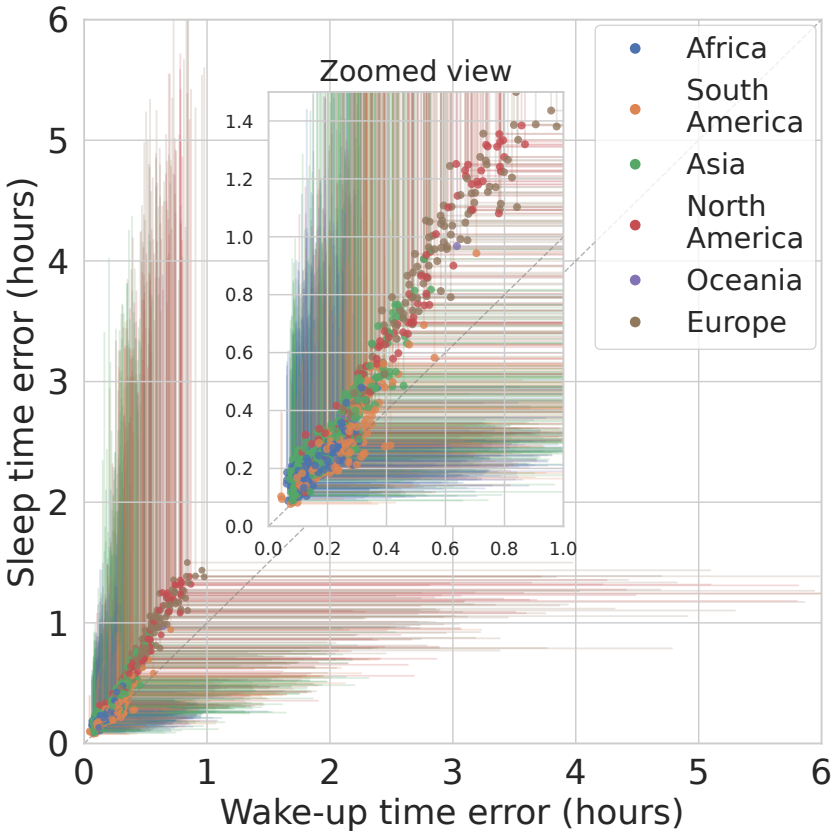


Figure 5.2: Distribution of the absolute prediction errors of wake-up and sleep time for every location, on x and y axes respectively. Color corresponds to the location's continent, points correspond to the median of the absolute error, and vertical and horizontal tails extend until the 95th percentile. Locations at the Equator have low error overall, while poleward regions show very high worst-case errors.

We first observe that many locations exhibit relatively small errors overall, with medians below 30 minutes and 95th around 1 hour. This concerns most peers in Africa, and many peers in Asia and South America. On the other hand, temperate peers in Europe and North America tend to have much higher errors, with medians above 1 hour, and worst case reaching 5 hours, demonstrating the necessity to tolerate a significant uncertainty in our procedures with this choice of prediction method.

We also note that the sleep error is typically $\sim 40\%$ larger than the wake-up error, likely due to the inertia brought by energy storage, which yields more variable sleep times.

The high errors are expected, because our method is simplistic, as building accurate summaries were not the aim of this work but rather a mean to evaluate our protocols. More involved prediction methods will be proposed in chapter 6.

5.3 Discovery delays

In our first experiment, we want to understand how long it takes for joining peers to discover existing ones, because knowing enough peers is crucial to establish good replication sets. We simulate peers joining the system progressively at the rate of one every 4 hours, and measure the average percentage of peer addresses and summaries known by each peer through time. We do this for different values of Δ_d, which is the duration a peer waits between polls to a random known peer when it tries to join the system.

Figure 5.3 shows the results of this experiment. As expected, retrying more often makes global discovery quicker, enabling the other protocols to operate quicker as well. Decreasing the duration too much will incur a higher message passing cost, but increasing it too much has the same effect, due to the fact that gossiped peer information will take too long to arrive to all peers, and they will therefore exchange incomplete discovery messages several times.

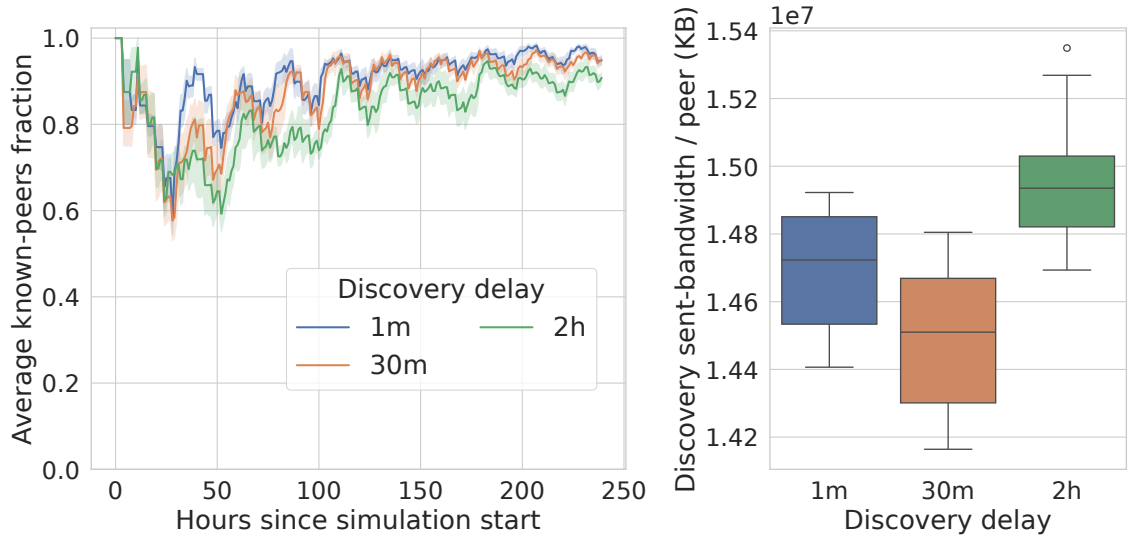


Figure 5.3: Impact of varying Δ_d. On the left plot, the different curves correspond to different values for Δ_d, the parameter corresponding to the delay between attempts to communicate with a random peer in the discovery routine. X-axis is the hours since the start of the simulation, y-axis is the average, among all peers in the system, of the fraction of all peers they know. The right plot shows the communication bandwidth cost for the different values of the parameter, as a box plot. A new peer joins every 4 hours. Using a short delay improves discovery speed, but can impact bandwidth use.

Interestingly, we also notice, aside from the bumps every 4 hours, regular oscillations on the scale of 24 hours. We postulate this is caused by the difference in the number of available peers at around 09:00 to 14:00 UTC, when most landmasses are in daytime, versus at 00:00 UTC, when the sun is mostly above the Pacific Ocean. We will go into more detail in section 5.7,

Setting the delay to 30m appears a reasonable compromise, and we used this value in remaining experiments. In all cases and in all of our tests, peers successfully discover each other within 1-2 days at worst.

5.4 DTN routing

We evaluate our DTN communication protocol, using the most common metrics for DTN routing:

- Delivery time, more specifically, the delay with respect to the Earliest Possible Delivery time (EPD)
- Hop count, or the difference with the minimum hop count.

The difference with EPD has a direct impact on the speed at which replication sets can converge, while minimizing the hop count reduces the load experienced by relays.

Assuming p sends a message to q at time t, we define EPD as the earliest time t' after or equal to t at which q is awake. Obviously, q could not have received the message before, and assuming there is a covering set of peers in the system, there must exist a space-time path going exclusively through awake peers which ends at q at t'. We compute the minimum hop count by recording the actual sleep and wake-up times of all peers in the system during simulations, and successively jumping from p as late as possible to the awake relay which stays awake the longest, until reaching t'. In some instances, if a message's delivery is worse than EPD, it can achieve smaller hop count than the corresponding optimum (e.g., the sender waits until the next day to send a message directly instead of forwarding it). In our analysis, we consider delivery time as the most important metric.

We simulated 200 peers all joining the system simultaneously, and sending dummy messages to each other at random using the DTN routing protocol. As explained, in our DTN protocol, relays try to send pending message to their destination when it is likely awake. This is parametrized by Δ_r, the time between successive retries when the destination does not answer, and Δ_g, the duration before and after the predicted wake-up and sleep times of the destination, when the relay attempts to contact it.

Figure 5.4 shows the impact of varying Δ_r. As intuitively expected, decreasing it dramatically affects the message passing cost (both bandwidth and count), but significantly decreases the difference with EPD. The relationship with the difference with EPD appears linear, for all three measured delay quantiles, which is not surprising. Polling twice as frequently will reduce by two the duration relays are not polling, and the difference with EPD happens in that time-span. On the other hand, it has positive but negligible impact on hop count, as the difference with the minimum hop count is already very small.

Figure 5.5 varies Δ_g instead, and displays a very similar trade-off between EPD and message passing cost, and with a more noticeable impact on hop count.

This dramatic positive impact on 90th and 95th percentiles of contacting peers up to 1 hour before the usual time they wake up is due to the high variance some locations exhibit, especially in high-latitude locations during winter, as we saw in 5.2.

Interestingly, all quantiles stop decreasing between 50 and 100 minutes, but the mean

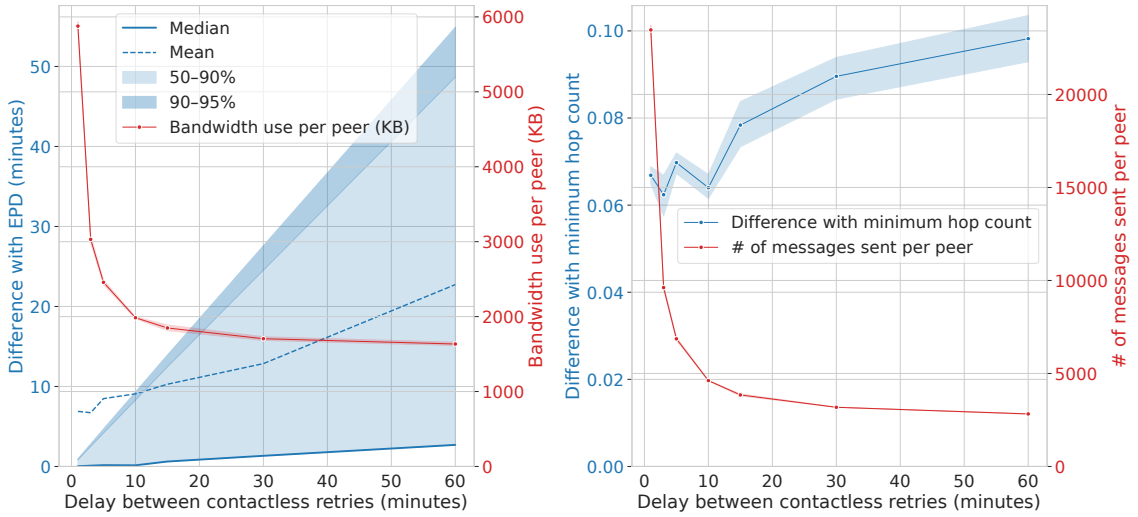


Figure 5.4: Costs and benefits of varying Δ_r (i.e. the duration between attempts to send a message to a peer in the DTN protocol). Both graphs vary Δ_r in minutes in the x-axis. Left graph shows, on the left y-axis, the difference with EPD in minutes, including mean, median and quantiles shown as shaded regions, and on the right y-axis, the total bandwidth usage per peer. Right graph shows the average difference with the minimum hop count and the total number of messages sent per peer on left and right y-axes. Low duration improves delay with EPD and hop count but increases message passing cost.

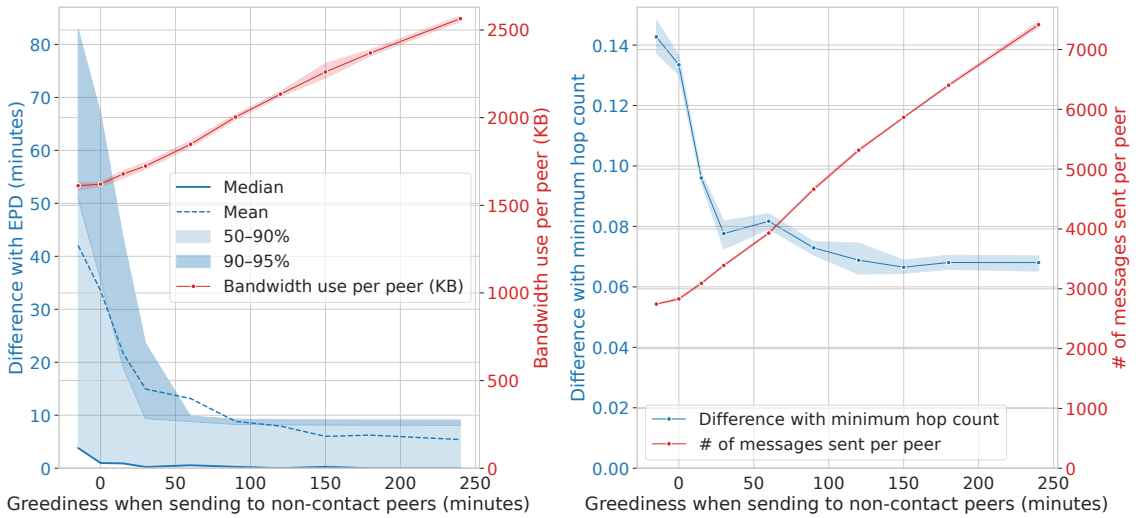


Figure 5.5: Costs and benefits of varying Δ_g (i.e. the time before the expected wake-up of a peer when relays start to attempt to send it a message in the DTN protocol). Both graphs vary Δ_g in minutes in the x-axis. Left graph shows, on the left y-axis, the difference with EPD in minutes including mean, median and quantiles shown as shaded regions, and on the right y-axis, the total bandwidth usage per peer. Right graph shows the average difference with the minimum hop count and the total number of messages sent per peer on left and right y-axes. Higher greediness improves delay and hop count but increases message passing cost.

continues to decrease. This is caused by very rare, very high delay outlier differences with EPD, that even cause the mean to be briefly above the 95th percentile. Those differences correspond to when a peer p sends a message to q at t, just before q goes to sleep later than usual. Since q was awake at send time, EPD is at t. But, due to insufficient Δ_g, no attempt was made to transmit the message to q, which means it will only be delivered when q wakes up the next day, potentially many hours later, causing the large difference with EPD. Increasing the greediness decreases the chances of this happening.

We also note that the median is very low even with very low Δ_g. The cause is that many messages can be sent immediately and directly, without needing to go through the DTN. As peers can be awake for 12 hours or more per day, often direct communication is possible, and in this case, EPD is easily obtained.

Given our system usually operates at the scale of hours, we consider than being within $5-10$ minutes of EPD in most cases is satisfying, and therefore the graphs suggest than setting Δ_r and Δ_g to 10 minutes and 60 minutes respectively strikes a good cost-benefit balance in our specific setup.

5.5 Finding replication sets

We proceed with our actual replication routine. The main goals are that replication sets are highly available, of relatively small size, and that they converge in bounded time.

5.5.1 Redundancy and continuous availability

We first evaluate the distribution of the availability of replication sets through time. We are interested, at time t, to understand for each peer p, how many of its hosts are awake compared to it is desired replication factor d_r. While each peer p considers their *believed* redundancy to choose replication hosts, which is computed using their summary, we evaluate instead the *actual* redundancy, which is the real number of its hosts awake at t, unknown to p. Naturally, to ensure the continuous availability of p's service, it must be ≥ 1.

As before, we simulated 200 peers starting simultaneously, with desired replication $d_r = 2$, and the distribution of the redundancy with safety margin $\Delta_m = 0$ is shown in figure 5.6.

The median redundancy is consistently at or above the desired value of 2. At regular intervals, many peers have more hosts than d_r, and we observe a periodic oscillation pattern, every 24 hours, with peaks and valleys. As suggested above, we understand this is caused by the skewed geographic distribution of peers. In particular, peaks correspond to Eurasia and Africa while valleys correspond to the Pacific Ocean, where it is harder to find hosts.

Looking at the worst case behavior, we see that tail peers frequently fail to attain redundancy of d_r, and that it even drops to 0 on rare occasions, which constitutes a failure of the continuous availability objective.

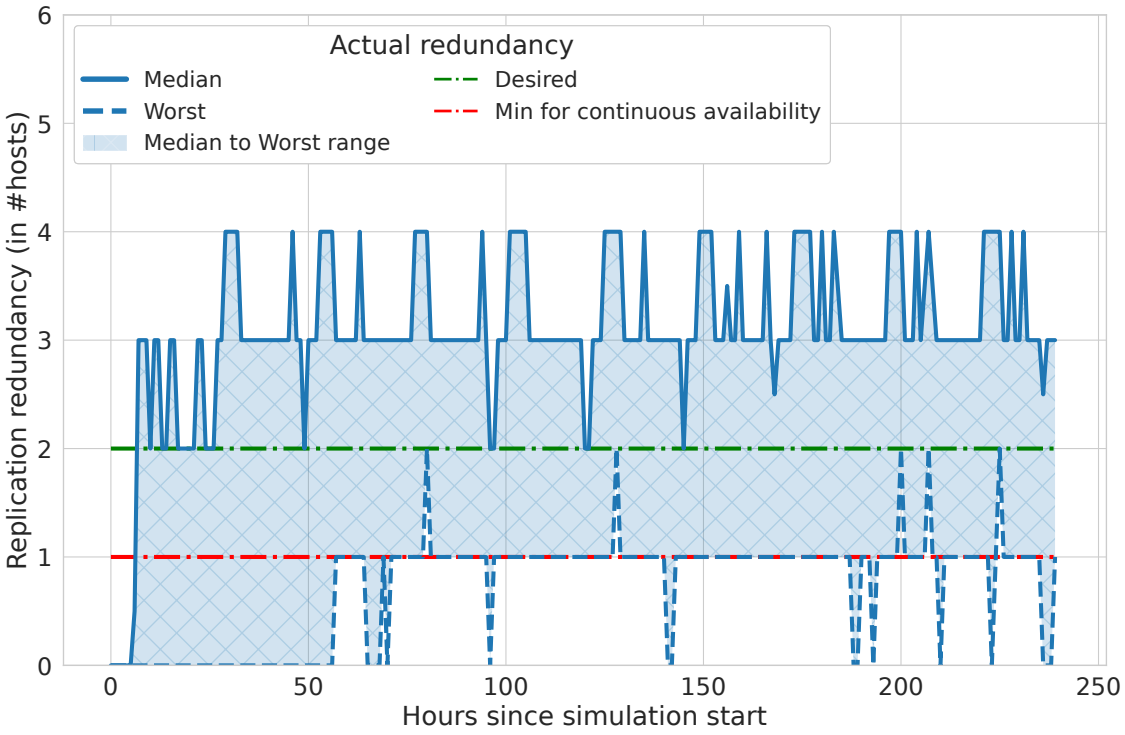


Figure 5.6: Evolution over time of the actual instantaneous awake host counts, without safety margin. The solid line corresponds to the median among simulated peers, the dashed line corresponds to the worst instantaneous redundancy of all peers, and the shaded region visualizes the range between them. The green line is the desired redundancy d_r and the red line is the minimum redundancy to ensure continuous service. While the median is consistently above the desired redundancy, the worst replication sets regularly breach the desired objective, and occasionally fail to achieve continuous service.

The simple reason is that the summary errors discussed in 5.2 are enough to make it possible for several hosts to have a shorter availability window than expected, causing some peers to have 0 hosts available in some instances. This is the reason we introduced the margin parameter Δ_m, and figure 5.7 shows the impact of using Δ_m = 1h instead.

After about 100 hours, the time for every peer to reach believed redundancy of d_r, the actual redundancy never drops to 0, showing the effectiveness of this change. However, the median values increased, and naturally we expect that this improved redundancy has a cost on the number of hosts.

Figure 5.8 confirms it, and setting the margin to 2h even led to a dramatic increase.

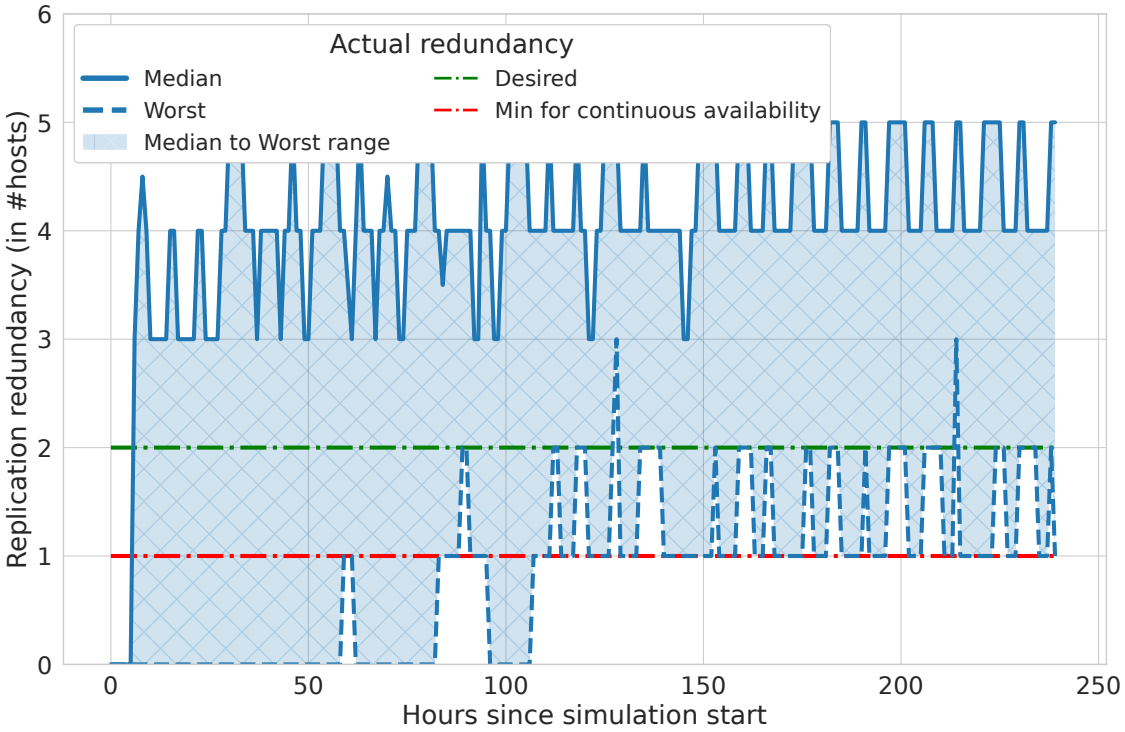


Figure 5.7: Evolution over time of the actual instantaneous awake host counts, with a safety margin of 1 hour. The solid line corresponds to the median among simulated peers, the dashed line corresponds to the worst instantaneous redundancy of all peers, and the shaded region visualizes the range between them. The green line is the desired redundancy d_r and the red line is the minimum redundancy to ensure continuous service. After convergence, all peers achieve continuous service, and the desired redundancy is satisfied more consistently.

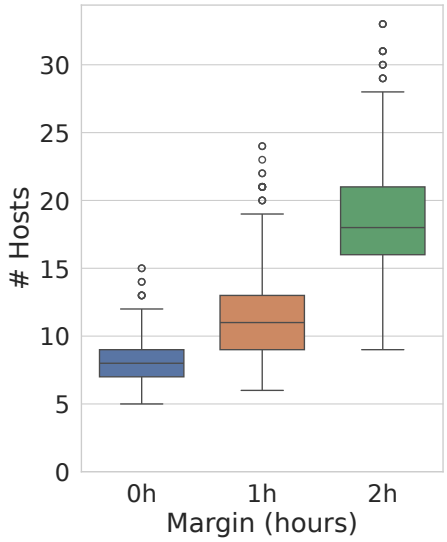


Figure 5.8: Distribution of the number of hosts after convergence, using margins of 0, 1 hour and 2 hours, as a box plot. Increasing the margin increases the number of hosts.

5.5.2 Convergence delay

Then, to understand more specifically the time it takes for peers to form satisfying replication sets and what influences this delay, we studied the impact of the 2 most important parameters, the host sampling temperature ρ_h and the host request delay Δ_h.

Using $\rho_h = 0$ corresponds to always requesting the peer with the highest complementarity score, while increasing it makes peers with lower complementarity more likely to be sampled. Setting it to a high value can help to relieve congestion by spreading out requests between peers with similar availability profile, and make the sampling more robust to peers not answering to requests, which can help to reach good coverage faster, but to the risk of forming suboptimal links and having more replicas than necessary.

We once again simulated 200 peers starting simultaneously, and measured the time they take to find satisfying hosts, as well as average the number of hosts they find. Figure 5.9 displays the above trade-off very clearly: Increasing ρ_h monotonously increases the number of hosts upon convergence. The lowest temperature values yield the worst convergence, but the quickest-converging value is not the highest temperature. Indeed, sampling peers with poor complementarity means that many more links need to be formed, to the point it slows down convergence instead of making it easier.

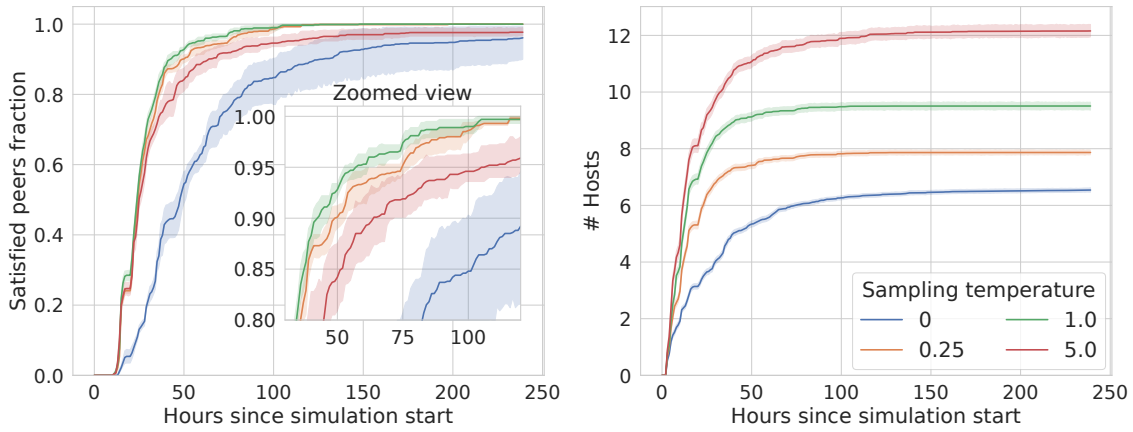


Figure 5.9: Cost and benefit of varying ρ_h (the softmax temperature used when sampling potential hosts based on their complementarity score). Left graph shows the hourly percentage of peers who found a covering set of replicas, for different temperatures. Right graph shows the hourly mean number of hosts, for the same set of temperatures. Increasing temperatures increases the number of hosts, but can improve convergence.

Setting $\rho_h = 0.25$ appears a good compromise, and this value was used in the rest of the experiments.

Varying Δ_h shows similar trade-off: We expect that waiting longer for replies from potential hosts will slow down the convergence of replication sets, but it could also ensure that new requests are made only when peers actually refused or will never answer requests, and can take into consideration the set of hosts which accepted in the meantime, in turn reducing the final number of hosts. Figure 5.10 clearly shows this effect: again, the number of hosts is

monotonously increasing when Δ_h decreases, but the time until convergence responds less visibly, except when it is so short (Δ_h = 3h) that convergence cannot be reached. Forming replication sets require asking diverse peers, some of which will not be awake for several hours. Requesting too often will hence not improve convergence, and can even harm it: early awake peers make so many requests that hosts available at crucial times reach their g_{max} limit, and refuse successive requests, making it impossible for late peers to find hosts.

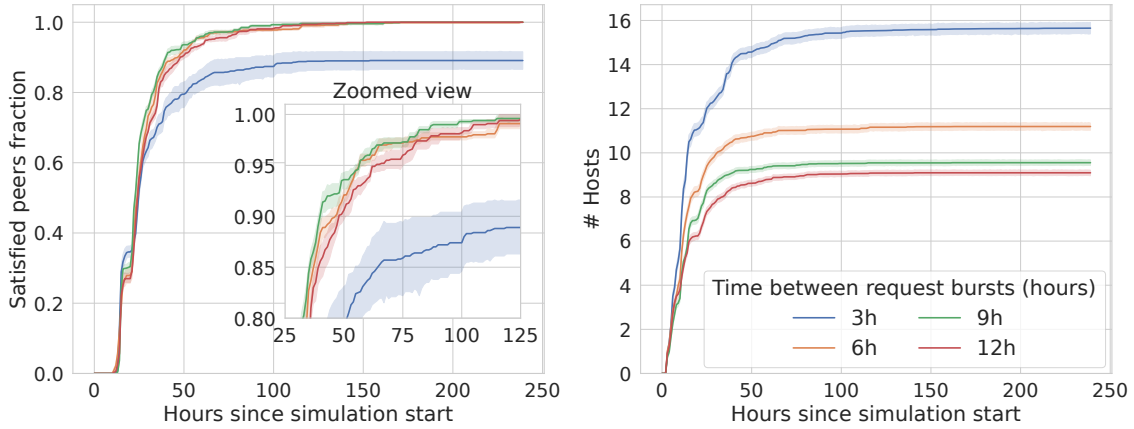


Figure 5.10: Cost and benefit of varying Δ_h (i.e. the delay between successive requests to potential hosts). Left graph shows the hourly percentage of peers who found a covering set of replicas, for different Δ_h. Right graph shows the hourly mean number of hosts, for the same set of delays. Lower delays cause higher host counts, and balanced delays improve convergence.

For most sets of parameters studied, we observe that all peers very reliably find a satisfying replication set (at least in this experiment, where there was diverse enough peers, and high enough g_{max} parameter), but it takes several hours, and in many case upwards of 50 hours. This is an expected consequence of the fact that every message must go through the DTN communication layer, which as we saw, can incur round-trip delays of 24 hours in the worst non-degenerate cases. Hence, if finding a satisfying set for some peer is done in 3 sequential steps (i.e., requests that depend on the result of the previous one), time to satisfaction of 60 hours or more are possible. The implications of these large delays will be explored in 6.

5.5.3 Greedy sampling

To evaluate the usefulness of our greedy sampling strategy, we compared it against an energy-blind variant which works the same way, except it samples the peers to ask at random instead of using their complementarity scores. This is achieved by setting ρ_h to $+\infty$. We also set d_r to 1, which lets us measure how many hosts are required for simple coverage. The resulting number of hosts after convergence are shown in 5.11. Our strategy performs much better, especially at the tail, for the highest quartile and outliers. This confirms the value of using energy profiles to replicate resources efficiently.

We note that the random sampling variant is not completely energy-blind, as it stops making new requests when it estimates its current set has sufficient complementarity. A completely energy-blind protocol that had no knowledge of peers availability, for example a standard DHT

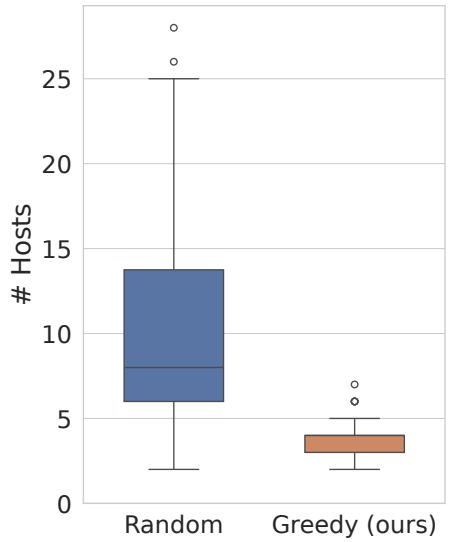


Figure 5.11: Distribution of the number of hosts per peer as a box plot, comparing our greedy sampling mechanism, against one sampling random peers until satisfaction. The black horizontal bars show the median, the box shows the interquartile range, the whiskers show the range of the data, except outliers shown as points. Greedy sampling performs much better than random sampling.

looking to replicate data among peers in our setup, would need to get enough random hosts such that the probability they do not have sufficient complementarity is very low, which would likely be even more hosts than the random variant shown here.

We also emphasize that those results depend on the specific energetic and geographical assumptions we made regarding servers, and that sets of servers with different availability profiles could benefit from our greedy algorithm to a variable extent.

5.6 Scalability

In our stated goals, we mentioned the cost, and in particular the message passing cost (number of messages sent per peer and total sent bandwidth per peer) should be constant in the number of peers. We studied the impact of increasing the number of peers on this message passing cost, from 25 to 800 peers, all starting simultaneously and being active for 10 days, giving them time to reach a satisfied state in each sub-protocol.

The results are shown in figure 5.12. Message count and bandwidth both show comparable results, which is that all message types achieve the desired constant scaling, except for "Known summaries statuses", the only message type which contains lists of all the currently known addresses and summaries of a peer. As in our discovery protocol, each peer aims to discover every other peer, discovery message sizes grow linearly with the number of peers. Clearly, our discovery procedure is the main reason preventing our system to truly scale, and we will propose improvements in 6.

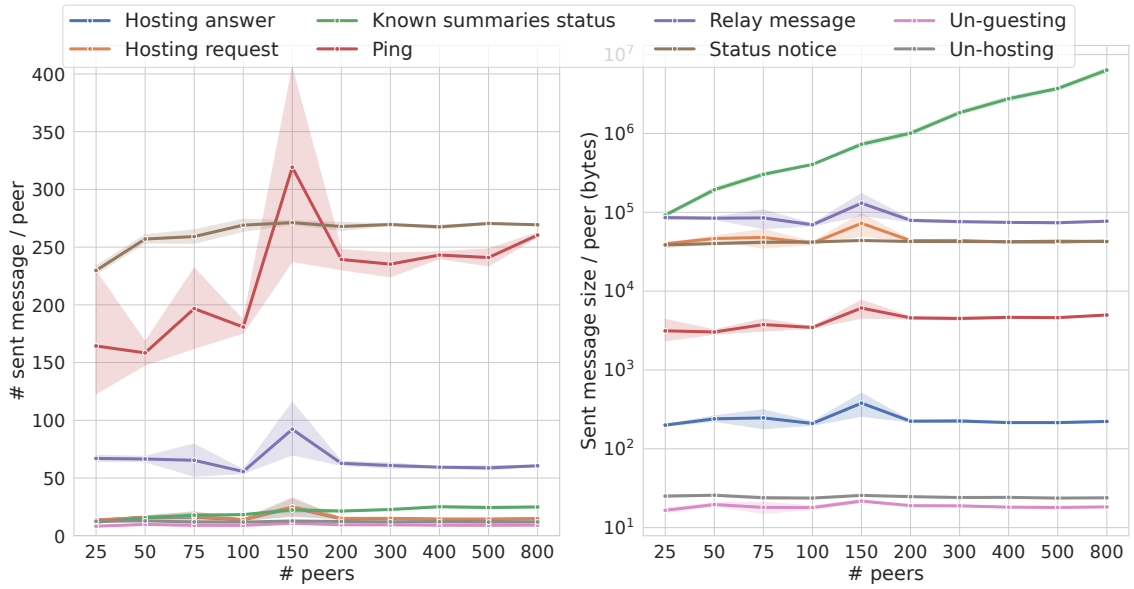


Figure 5.12: Message passing complexity in function of the number of peers, for different message types corresponding to different sub-protocols. Left shows the average number of messages sent per peer, while right shows the average total sent size, or bandwidth used by each message type, per peer, in logarithmic scale. All message types scale constantly except `KnownSummaries`.

In any case, the total used bandwidth, not exceeding the magnitude of 10MB per peer for 800 peers, corresponds to the aggregate usage over 10 days, or about 50KB per hour, which is negligible for most systems, and remains small even for today's very low-power or embedded systems, and for communities with very low-bandwidth connectivity. For most realistic services one would want to host and replicate, the cost of moving data related to the service between replicas would dwarf the cost of our protocols maintaining the set of replicas.

5.7 Heliotropism and geography

The goal of this section is to evaluate the extent to which our system adapts itself to geographical realities, the impact those realities would have if such a system was deployed, and whether our systems load, traffic and structure *follow the sun*, which was part of our stated goals.

We simulated 400 peers starting at the same time, and operating normally for an entire year, with relatively tight guest constraints, in order to see which region of the world would experience the highest load. Figure 5.13 shows the regions of the world having more contacts than most others, in red, at 4 snapshots in the year. January and July roughly correspond to periods with the irradiance maxima and minima in the northern and southern hemispheres, while in April and October energy is more equally distributed.

In every season, Oceania and parts of the western side of the Americas have a very high load. The main reason for this is the size of the Pacific Ocean, inside which there are few if any servers running due to land-surface uniform sampling of locations, which creates bottlenecks.

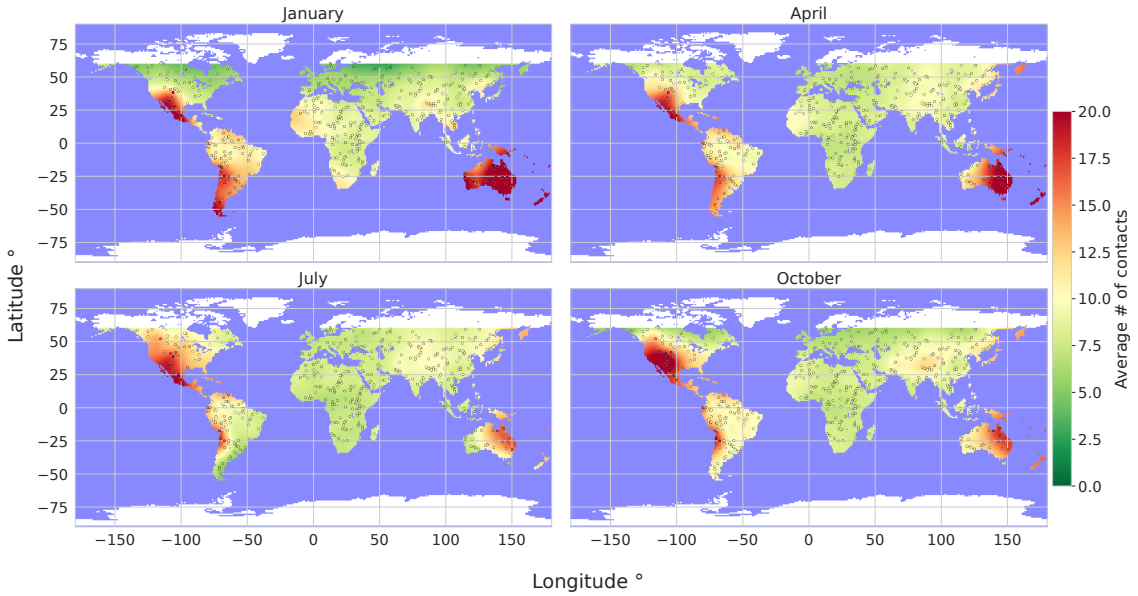


Figure 5.13: Average number of contacts represented as a color from green (0) to red (20), for all servers in a large simulation lasting an entire year, RBF-interpolated onto the map of the world. Highest load is concentrated on the edges of the Pacific Ocean, and reacts to seasons.

For example, a server in eastern Brazil holding a message to relay will not be able to forward it directly to the first server available across the Pacific, in Oceania. Hence, it will choose the server in the Americas that will stay awake the longest, typically in Patagonia in January, or in California in July. All the servers in a large set of longitudes will rely on those few servers, which essentially become choke points.

On the other end, the only servers that can receive this message will be located in Oceania, especially in January, as East Asia's servers will not be available early enough, causing all servers in Australia to be completely overloaded in this season. A secondary cause to this imbalance is the relative lack of land surfaces on the Southern Hemisphere, compared to the Northern Hemisphere (only 24% of earth's land surface, excluding Antarctica and Greenland).

Conversely, servers located in Western Asia or Africa, despite often receiving abundant energy, will not suffer from high contact pressure, because they are surrounded by large land masses east and west, with many servers capable of relaying messages passing through the region during any season.

Figure 5.14 presents a different view of this imbalance, and shows the relationship between the duration of the daily availability window of a peer at some time in the year, and the amount of contacts it has at this moment. The points are clearly bounded in a triangle region: For the most useful points such as those in Oceania or North America, their number of contacts is noticeably growing in function of their available hours. Inversely, peers in Africa and Asia, despite having a lot of available hours, only have barely more contacts as they need for themselves (about 6-7 in this simulation), and are all clustered in the bottom right.

Similarly, figures 5.15a and 5.15b show the average number of hosts and guests in the same experiment.

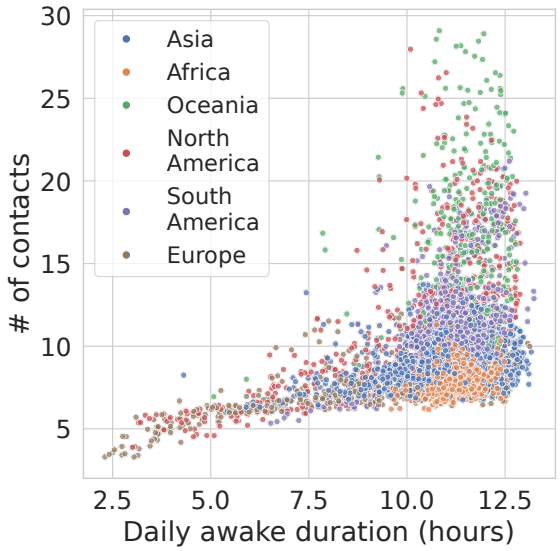


Figure 5.14: Relation between the number of daily availability hour and the number of contacts, snapshots taken at different seasons at many locations, and colored by their continent. Available peers do not necessary have more contacts, but unavailable peers do have less.

Differences in host counts (5.15a) are small, which is expected, given that they can all ask peers around the world with high complementarity to host them, no matter where they are located. We observe a trend with peers in Canada and Siberia, or peers in rainforests receiving the least energy or the highest energy variability having slightly more hosts to compensate for it.

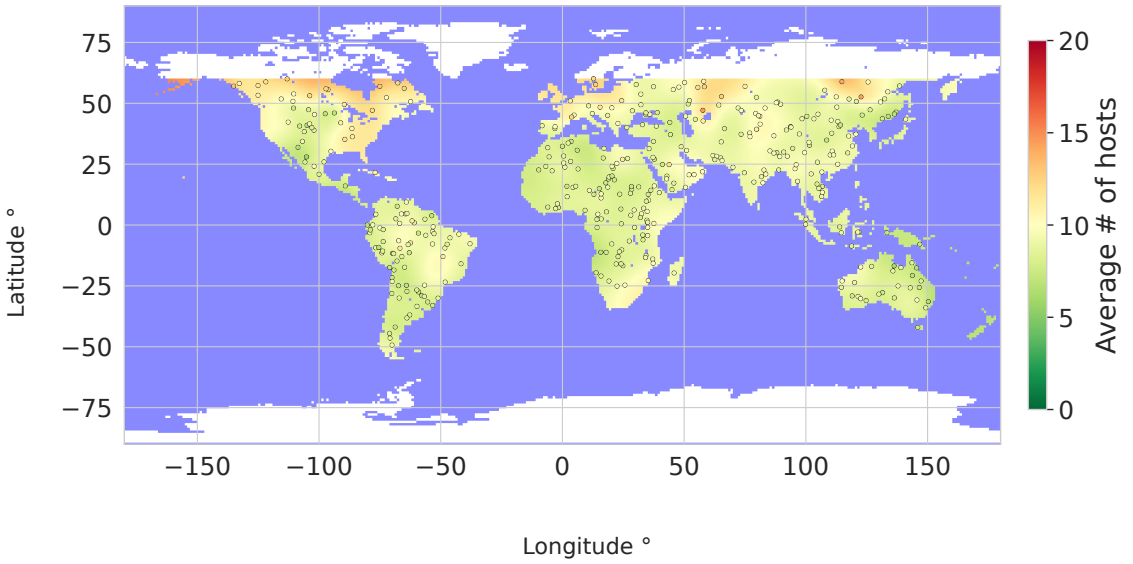
However, differences in guest counts (5.15b) are drastic, exhibiting a pattern comparable to the contact counts but more pronounced. Many peers in Africa and western Eurasia have few guests, meaning they appear useless and redundant for most peers trying to find good replicas. On the other hand, peers in regions like western North America, Australia and parts of East Asia are saturated, for the same reasons presented above, mainly the size of the Pacific Ocean.

Finally, our third map in figure 5.16 shows, for a similar experiment with 400 peers running normally for the challenging month of January, the average number of messages relayed by each location. This includes only messages for which the location is neither the sender nor the receiver, and hence studies the potential routing choking points.

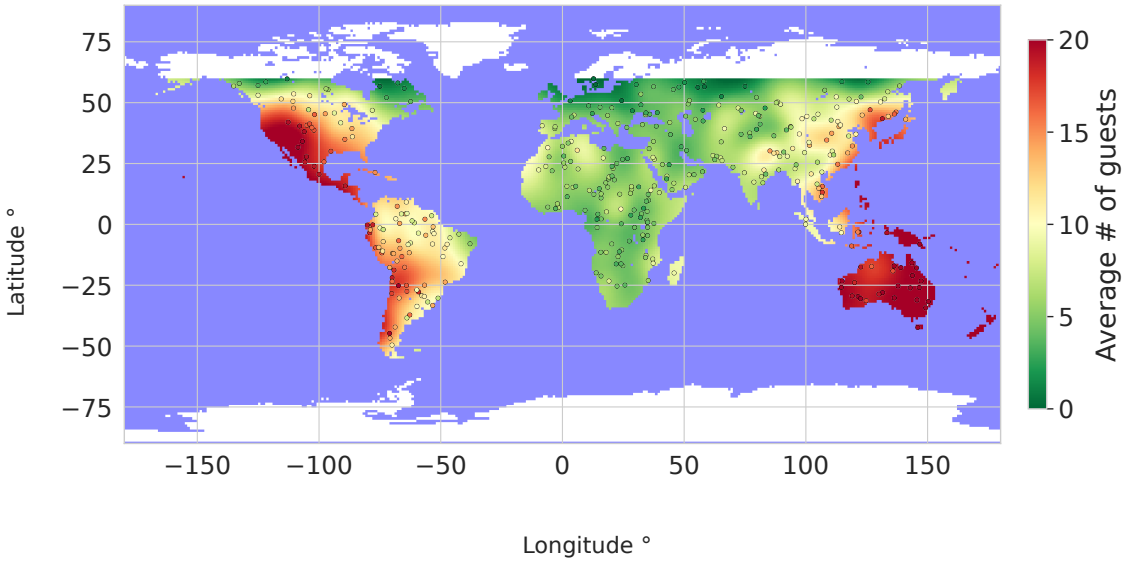
The striking result is an imbalance in load in the same areas as earlier, but to a much steeper extent. Oceania and the very south of the Americas appear again to hold significant load, and a few specific peers in Western Mexico seem to be chosen as relays for a large portion of the messages passing in this region. In general, load seems to be concentrated on the Western side of continents.

This suggests the need for a mechanism prompting peers not to choose the best single relay among their contacts every time, which is often possible due to existing slack in window overlaps, to better spread out the load between relays, at the cost of higher hop counts in some cases.

In general, these geographic experiments highlight the sensitivity of the system to the specific locations of peers in the network. If there were much fewer peers in Australia, it is not guaranteed



(a) Number of hosts



(b) Number of guests

Figure 5.15: Average number of hosts and guests, represented as a color from green (0) to red (20), for all servers in a large simulation, RBF-interpolated onto the map of the world. Host counts are evenly distributed, but guest counts are again heavily concentrated to the Pacific's edges.

that the DTN routing and replication routines would have succeeded. On the other hand, if a non-negligible portion of peers were located on islands in the Pacific, this could have significantly reduced the congestion we observed.

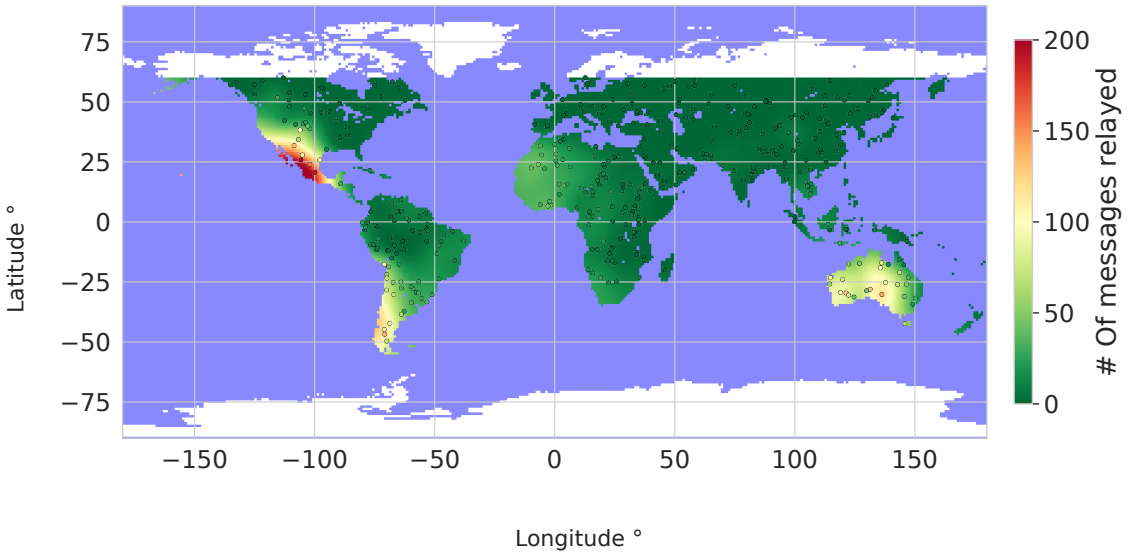


Figure 5.16: Average number of relayed messages for each server (i.e., the location was neither the source nor the destination), represented as a color from green (0) to red (200), in a large simulation lasting for the month of January, RBF-interpolated onto the map of the world. A few Mexican servers relay a large part of traffic.

Chapter 6

Discussion

This chapter discusses the limitations of our approach and modelling, presents some implications of the results, and proposes avenues for future research.

6.1 Limitations

Due to time constraints and the breadth and depth of considerations which emerged in the realization of this work, several relatively strong simplifying assumptions have been made, and our approach has notable shortcomings.

6.1.1 Modelling

As mentioned, our energy harvesting and consumption model are simplistic. We assumed linear charging, while real energy storage units tend to have more complex power-to-charge functions, exhibiting exponential features [58]. This does not change the underlying setup, being that servers have roughly predictable daily availability windows, but a better model could give more empirical weight to the simulated performances.

We also modelled consumed power as a binary constant (awake) or zero (asleep), while in practice, an idle server consumes much less than a fully loaded one. It is likely that the energy consumption of a host would significantly increase if it had more guests, which would in turn reduce its availability window and as a result cause guests to discard the link, and a more precise model would be needed to observe and analyze this phenomenon.

6.1.2 Failure and threat model

We wanted to focus on tackling the challenge of intermittence, so we adopted a very optimistic failure and threat model. We assumed no peer is byzantine, which in practice often implies that every node needs to be fully trusted, and we did not explicitly handle peer crashes.

Most approaches we envision to make the system robust to various failures involve increasing redundancy targets across sub-protocols. Indeed, across every operation in our protocols, no single specific node or no quorum fundamentally needs to be trusted. Therefore, having more contacts and replication hosts than strictly necessary, cross-checking summary information across different gossipers, and forwarding each relayed message across multiple disjoint routes could be the first steps in making the systems more resilient.

6.1.3 Discovery mechanism

As was made clear in section 5.3, our discovery mechanism prevents our system to scale gracefully. A very different approach is needed for 10^6+ peers to be able to join the system and establish replication links without discovering, storing and considering information about that many peers.

An option is for each peer to maintain a partial *view* of the system, which must be large and diverse enough for complementary sets to be possible, but small enough to be cheap to maintain. According to our experiments and with our energy assumptions, knowing a diverse set of 100 to 200 peers already enables strongly available sets to be formed relatively easily. Protocols for partial gossip membership view like HyParView [27] give each peer a partial view of a large gossip network, and could replace our discovery mechanism.

6.2 Practical considerations

As explained in chapter 4, our core peer implementation was designed to use abstract time, energy and network modules, to make the simulation code as similar as possible to the code we would use in actual deployments. However, a few remaining details would need to be modified in this case.

- First, if sleep operations correspond to actual shutdowns of the server, the entire state needs to be written to disk, and loaded back upon waking up.
- Second, the underlying network would ideally provide reliable communication primitives in case both peers are awake. In real situations, it is not obvious how a sender can distinguish a peer not answering because it is sleeping, or because packets got lost. However, the time-scales relevant in both cases are very different. In case of packet losses, retry should happen within a fraction of a second, while for sleeping peers, retry should not happen before several minutes. This fact can inform the design of a TCP-like reliable primitive, that would make a few attempts at direct communication, and report that the peer is likely sleeping if no response arrived.

6.3 Insights

This section presents insights that emerged when we implemented and evaluated the system.

6.3.1 Time and delays

In the system we proposed, the order of magnitude of delays and timings is very high, minutes, hours or even days, which has both technical implications and usability consequences.

First, all the delays pertaining to usual IP networking, ranging from seconds to milliseconds, become completely irrelevant in most operations. This eliminates many problems arising in low-latency environments, and in practice, our event-based system, sending only messages at a few occasions per hour, can hardly ever cause or experience network congestion.

Then, as an obvious consequence of the DTN, any update to the system can take days to propagate. This means that peers having an insufficient replication set, for example, could take hours to react to new arrivals and improve their coverage. This contrasts with today's systems, which tend to offer and value immediate reactivity. Instead of viewing this as a problem, we see it as a natural consequence of using ambient and renewable energy, which is acceptable for many applications that don't require immediate updates such as blog publishing, in a similar spirit to principles put forward by the *computing within limits* community [1, 13, 51, 55].

6.3.2 Incentives

Beyond the question of resilience to byzantine actors in 6.1.2, implementing the proposed system in practice would require a more explicit handling of the incentive mechanisms at play. In our work, we assumed benevolent and selfless participation, which is reasonable only if all nodes are controlled by a single entity, and is unrealistic and undesirable for geo-distributed off-grid nodes, or if nodes can only join through a trusted process. For example, nodes joining the solar protocol [51] are manually added by the network moderators.

If many unrelated and non-trusted nodes join the network, an incentive might be required to ensure they accept to host other peers services. A mechanism resembling BitTorrent's tit-for-tat [12] could be explored. If we define the number of replicated hours as the number of hours a peer was available, multiplied by the instantaneous amount of services it was replicating, we may pose that this peer deserves the same number of replication hours from other peers. This means a very available peer would not need to replicate many services to merit its own replication quotas, while a northern peer in winter would need to replicate many services during the few hours it is available, to be able to ask more available peers to replicate its services.

The problem in this approach is that some locations are simply more useful than others, as was seen in the global heatmaps. A peer located in Copenhagen, surrounded by other peers offering the same availability would struggle to get enough replication quotas to deserve to be replicated by an isolated peer on a Pacific island, which would receive requests from much more

peers. To ensure each peer gets replicated by enough hosts, a quantity of non-reciprocal, selfless hosting should be accepted, or other incentives mechanisms, such as remuneration could be envisioned.

6.4 Future Work

Due to the lack of research on geo-distributed fully renewable peer-to-peer systems, many questions remain open, and we propose incremental extensions to our proposal, as well as more ambitious future work directions.

6.4.1 Direct improvements

Online decisions

Our protocols make decisions using exclusively predictions from the `Summary` of peers, which are essentially types of climate profile, averaged over several years and without any knowledge of the current weather conditions. Successive days weather is usually strongly correlated, and measuring, reporting and communicating recent energy and status observations between peers could improve the accuracy of their routing and contact choices. Going further, peers could even incorporate weather observations and predictions coupled with solar harvesting models to get even more accurate predictions of their own and other peers expected wake-up times [54, 61, 62].

Summaries as probability distributions

As was evident from the distribution of errors in figure 5.2, some peers exhibit much higher uncertainty than others in their availability patterns. Modelling availability, not as a fixed time interval, but as a probability distribution could make several parts of the system more robust, efficient and elegant.

For example, routing paths could be chosen to maximize the probability that the message is delivered at EPD time and minimize the expected number of hops, replication sets could be chosen to maximize the probability that at least one peer in the set is available, contacts could be chosen so that the probability of daily overlap is close to 1, and the expected number of available hours after the message is transmitted is maximized.

A challenge with this approach would be to ensure that the correlation between location's energy availability is considered correctly: Indeed, peers located relatively close tend to exhibit high correlation in their day-to-day availability due to the large scale of macro-weather patterns, and therefore, treating them as independent would be incorrect, for example when computing the probability that at least one of them is available. This could require that summaries include covariance information regarding other summaries, and hence that they should be cooperatively

computed.

6.4.2 New directions

Finally, we propose chosen avenues for future research, which further differ from our proposal.

Forming bidirectional and clique replication links

Our replication links are unidirectional, and they form a directed graph. However, in many cases, complementarity is approximately symmetric, particularly for peers having opposite longitudes or latitudes. Choosing to form bidirectional links (i.e. undirected edges) could reduce communication overhead as nodes would need to communicate with fewer peers, and it could act as a natural incentive mechanism based on reciprocity. However, it would likely be harder to form stable links, as both parties need to be interested in the link instead of just the requesting peer.

Going further, the replication links could form cliques, or clusters. Each peer in the cluster would replicate the services of every other peer in the cluster. Because the cluster would be chosen to be highly available, it could in principle be used to route requests to other members of the cluster efficiently, partially circumventing the need for the usual DTN protocol. However, it would be even harder to form great clusters, as all peers in the cluster need to be complementary. A peer with low-availability would find it hard to join existing clusters, unless a margin is reserved for selflessly accepting peers in the cluster even if their availability window is useless to current members.

Using a structured overlay

If a structured peer-to-peer overlay which allows similarity search between availability vectors or availability windows was used, it could be possible to find highly complementary peers using this structure, preferably in $O(log(n))$ sequential requests in the number of peers or less, instead of relying on local knowledge of hundreds of peer summaries. Such a feature would remove the need for a discovery mechanism.

Aside from finding or designing a suitable overlay, the main difficulty we envision is that the requests need to rely exclusively on awake peers: If queries relied on arbitrary peers, they could take $\Omega(log(n))$ days in the worst case, as each sequential request would have to go through the DTN protocol. An approach could be to replicate the DHT itself in an availability-aware way, such that all the necessary information to complete any request is always held by some available peers.

Giving-up the battery

Finally, one could study the impact of reducing the energy storage capacity, up to the point where it is only sufficient to power servers for a few tens of seconds, just enough to enable graceful sleep instead of abrupt shut-down when energy becomes insufficient. Such storage could be offered by capacitors instead of regular Li-ion batteries. This would further decrease the environmental and economic cost of servers, but would likely invalidate our representation of daily availability as non-fragmented windows, and might impact significantly the performance of our protocols. Another, more general but potentially less efficient set of protocols and representations could be needed in replacement.

Chapter 7

Conclusion

This thesis investigates the possibility of organizing a set of intermittent, entirely solar-powered servers to replicate each-other's computing resource in a decentralized way, in the aim of allowing them to offer a continuous service. This is in large part motivated by the pressing need to stray away from fossil fuels, and as an option for the numerous communities worldwide that live off-the-grid, by choice or not.

To achieve this, we propose that each server forms replication links with complementary peers in terms of energy availability, using a greedy covering algorithm inside a peer-to-peer system. We also propose to meet communication needs in this context by simple and purpose-fit solutions for delay-tolerant routing and peer discovery, with the overarching aim to minimize message passing overhead, and to avoid busy-polling unavailable peers when unnecessary.

Results demonstrate the feasibility of running applications that require high-availability on periodically intermittent peers using much fewer replicas than an energy-blind baseline, but outline the dependence of such a system on geographical realities, and on the amount of servers and their specific locations. Trade-offs arise in such a setup, contrasting bandwidth cost with message delivery delays, replication cost with convergence delay, or replication cost with service availability guarantees.

This work's purpose is not to propose the best possible solution to every sub-problem identified, but rather to combine several ideas coming from the fields of submodular optimization, delay-tolerant routing and energy-aware computing into a coherent proof-of-concept.

Therefore, and due to the lack of attention on this specific setup, many avenues for future research are open, and include making the system more resilient to various threats, solidifying its theoretical foundations, exploring different replication graphs, and most importantly, identifying the technical and logistical challenges of actually deploying such a geo-distributed system.

Bibliography

[1] Roel Roscam Abbing. "'This is a solar-powered website, which means it sometimes goes offline': a design inquiry into degrowth and ICT". en. In: *LIMITS Workshop on Computing within Limits* (June 2021). DOI: 10.21428/bf6fb269.e78d19f6. URL: https://limits.pubpub.org/pub/lecuxefc (visited on 08/07/2025).

[2] Mohammad Ali Abdelkareem et al. "Environmental aspects of batteries". In: *Sustainable Horizons* 8 (Dec. 2023), p. 100074. ISSN: 2772-7378. DOI: 10.1016/j.horiz.2023.100074. URL: https://www.sciencedirect.com/science/article/pii/S2772737823000287 (visited on 01/08/2026).

[3] Tania Alhajj and Vincent Corlay. *Improved Contact Graph Routing in Delay Tolerant Networks with Capacity and Buffer Constraints*. arXiv:2410.15546 [cs]. Apr. 2025. DOI: 10.48550/arXiv.2410.15546. URL: http://arxiv.org/abs/2410.15546 (visited on 11/21/2025).

[4] B. W. Ang and Bin Su. "Carbon emission intensity in electricity production: A global analysis". In: *Energy Policy* 94 (July 2016), pp. 56–63. ISSN: 0301-4215. DOI: 10.1016/j.enpol.2016.03.038. URL: https://www.sciencedirect.com/science/article/pii/S0301421516301458 (visited on 12/29/2025).

[5] Kodami Badza, Marie Sawadogo, and Y. M. Soro. "Environmental impacts of a stand-alone photovoltaic system in sub-saharan Africa: A case study in Burkina Faso". In: *Heliyon* 10.19 (Oct. 2024), e38954. ISSN: 2405-8440. DOI: 10.1016/j.heliyon.2024.e38954. URL: https://www.sciencedirect.com/science/article/pii/S2405844024149857 (visited on 01/08/2026).

[6] Piotr Berman, Marek Karpinski, and Andrzej Lingas. "Exact and Approximation Algorithms for Geometric and Capacitated Set Cover Problems". en. In: *Algorithmica* 64.2 (Oct. 2012), pp. 295–310. ISSN: 1432-0541. DOI: 10.1007/s00453-011-9591-5. URL: https://doi.org/10.1007/s00453-011-9591-5 (visited on 01/08/2026).

[7] Katherine Calvin et al. "IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (eds.)]. IPCC, Geneva, Switzerland." und. In: (2023). Publisher: Intergovernmental Panel on Climate Change (IPCC). DOI: 10.59327/ipcc/ar6-9789291691647. URL: http://hdl.handle.net/1854/LU-01KAV2B3ZQ47W8ZHHDZ81WAMNM (visited on 12/29/2025).

[8] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. *Time-Varying Graphs and Dynamic Networks*. en. arXiv:1012.0009 [cs]. Feb. 2012. DOI: 10.48550/arXiv.1012.0009. URL: http://arxiv.org/abs/1012.0009 (visited on 11/05/2025).

[9] Deeparnab Chakrabarty, Elyot Grant, and Jochen Könemann. "On Column-Restricted and Priority Covering Integer Programs". en. In: *Integer Programming and Combinatorial Optimization*. Ed. by Friedrich Eisenbrand and F. Bruce Shepherd. Berlin, Heidelberg: Springer, 2010, pp. 355–368. ISBN: 978-3-642-13036-6. DOI: 10.1007/978-3-642-13036-6_27.

[10] Changbing Chen, Bingsheng He, and Xueyan Tang. "Green-aware workload scheduling in geographically distributed data centers". en. In: *4th IEEE International Conference on Cloud Computing Technology and Science Proceedings*. Taipei, Taiwan: IEEE, Dec. 2012, pp. 82–89. ISBN: 978-1-4673-4510-1 978-1-4673-4511-8 978-1-4673-4509-5. DOI: 10.1109/CloudCom.2012.6427545. URL: http://ieeexplore.ieee.org/document/6427545/ (visited on 10/08/2025).

[11] Chien-Ming Cheng, Shiao-Li Tsao, and Pei-Yun Lin. "SEEDS: A Solar-Based Energy-Efficient Distributed Server Farm". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45.1 (Jan. 2015), pp. 143–156. ISSN: 2168-2232. DOI: 10.1109/TSMC.2014.2329277. URL: https://ieeexplore.ieee.org/abstract/document/6847173 (visited on 08/10/2025).

[12] Bram Cohen. "Incentives Build Robustness in BitTorrent". en. In: ().

[13] Marloes De Valk. "A pluriverse of local worlds: A review of Computing within Limits related terminology and practices". en. In: *LIMITS Workshop on Computing within Limits* (June 2021). DOI: 10.21428/bf6fb269.1e37d8be. URL: https://limits.pubpub.org/pub/jkrofglk (visited on 08/07/2025).

[14] Ahmad H. Dehwah, Jeff S. Shamma, and Christian G. Claudel. "A distributed routing scheme for energy management in solar powered sensor networks". In: *Ad Hoc Networks* 67 (Dec. 2017), pp. 11–23. ISSN: 1570-8705. DOI: 10.1016/j.adhoc.2017.10.002. URL: https://www.sciencedirect.com/science/article/pii/S1570870517301749 (visited on 09/10/2025).

[15] Neda Edalat and Mehul Motani. "Energy-aware task allocation for energy harvesting sensor networks". In: *EURASIP Journal on Wireless Communications and Networking* 2016.1 (Jan. 2016), p. 28. ISSN: 1687-1499. DOI: 10.1186/s13638-015-0490-3. URL: https://doi.org/10.1186/s13638-015-0490-3 (visited on 10/08/2025).

[16] Doyle Gallegos et al. *Connecting Africa Through Broadband : A Strategy for Doubling Connectivity by 2021 and Reaching Universal Access by 2030*. en. Text/HTML. URL: https://documents.worldbank.org/en/publication/documents-reports/documentdetail/131521594177485720 (visited on 12/29/2025).

[17] *Global renewable capacity is set to grow strongly, driven by solar PV - News*. en-GB. Oct. 2025. URL: https://www.iea.org/news/global-renewable-capacity-is-set-to-grow-strongly-driven-by-solar-pv (visited on 12/29/2025).

[18] *Global Solar Atlas*. URL: https://globalsolaratlas.info (visited on 12/30/2025).

[19] *Global solar installations surge 64% in first half of 2025.* en-US. URL: https://ember-energy.org/latest-updates/global-solar-installations-surge-64-in-first-half-of-2025 (visited on 12/29/2025).

[20] Wedan Emmanuel Gnibga, Anne Blavette, and Anne-Cécile Orgerie. "Renewable Energy in Data Centers: The Dilemma of Electrical Grid Dependency and Autonomy Costs". In: *IEEE Transactions on Sustainable Computing* 9.3 (May 2024), pp. 315–328. ISSN: 2377-3782. DOI: 10.1109/TSUSC.2023.3307790. URL: https://ieeexplore.ieee.org/abstract/document/10227588 (visited on 12/30/2025).

[21] Moritz Gutsch and Jens Leker. "Costs, carbon footprint, and environmental impacts of lithium-ion batteries – From cathode active material synthesis to cell manufacturing and recycling". In: *Applied Energy* 353 (Jan. 2024), p. 122132. ISSN: 0306-2619. DOI: 10.1016/j.apenergy.2023.122132. URL: https://www.sciencedirect.com/science/article/pii/S0306261923014964 (visited on 01/08/2026).

[22] Lorenz Hilty. "Computing Efficiency, Sufficiency, and Self-sufficiency: A Model for Sustainability?" eng. In: s.n., June 2015. URL: https://www.zora.uzh.ch/handle/20.500.14742/108864 (visited on 01/03/2026).

[23] M.J. Islam, M.M. Islam, and M.N. Islam. "A-sLEACH: An Advanced Solar Aware Leach Protocol for Energy Efficient Routing in Wireless Sensor Networks". In: *Sixth International Conference on Networking (ICN'07).* Apr. 2007, pp. 4–4. DOI: 10.1109/ICN.2007.14. URL: https://ieeexplore.ieee.org/abstract/document/4196197 (visited on 09/09/2025).

[24] Manuel Jesús-Azabal, José García-Alonso, and Jaime Galán-Jiménez. "Communication in Isolated Rural Areas: A Comprehensive Review of the Alternatives to the Internet". en. In: *Gerontechnology V.* Ed. by Enrique Moguel, Lara Guedes de Pinho, and César Fonseca. Cham: Springer Nature Switzerland, 2023, pp. 11–21. ISBN: 978-3-031-29067-1. DOI: 10.1007/978-3-031-29067-1_2.

[25] R. M. Karp. "On the Computational Complexity of Combinatorial Problems". en. In: *Networks* 5.1 (1975). _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/net.1975.5.1.45, pp. 45–68. ISSN: 1097-0037. DOI: 10.1002/net.1975.5.1.45. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/net.1975.5.1.45 (visited on 01/02/2026).

[26] Rennie B Kaunda. "Potential environmental impacts of lithium mining". In: *Journal of Energy & Natural Resources Law* 38.3 (July 2020). Publisher: Routledge _eprint: https://doi.org/10.1080/02646811.2020.1754596, pp. 237–244. ISSN: 0264-6811. DOI: 10.1080/02646811.2020.1754596. URL: https://doi.org/10.1080/02646811.2020.1754596 (visited on 01/08/2026).

[27] Joao Leitao, Jose Pereira, and Luis Rodrigues. "HyParView: A Membership Protocol for Reliable Gossip-Based Broadcast". In: *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07).* ISSN: 2158-3927. June 2007, pp. 419–429. DOI: 10.1109/DSN.2007.56. URL: https://ieeexplore.ieee.org/abstract/document/4272993 (visited on 12/30/2025).

[28] Yunbo Li, Anne-Cécile Orgerie, and Jean-Marc Menaud. “Balancing the Use of Batteries and Opportunistic Scheduling Policies for Maximizing Renewable Energy Consumption in a Cloud Data Center”. In: *2017 25th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*. ISSN: 2377-5750. Mar. 2017, pp. 408–415. DOI: 10.1109/PDP.2017.24. URL: https://ieeexplore.ieee.org/abstract/document/7912680 (visited on 09/12/2025).

[29] Weiwei Lin et al. “A systematic review of green-aware management techniques for sustainable data center”. In: *Sustainable Computing: Informatics and Systems* 42 (Apr. 2024), p. 100989. ISSN: 2210-5379. DOI: 10.1016/j.suscom.2024.100989. URL: https://www.sciencedirect.com/science/article/pii/S2210537924000349 (visited on 10/02/2025).

[30] Anders Lindgren et al. “Probabilistic routing in intermittently connected networks”. In: *ACM SIGMOBILE Mobile Computing and Communications Review* 7.3 (July 2003). Publisher: Association for Computing Machinery, pp. 19–20. DOI: 10.1145/961268.961272. URL: https://dl.acm.org/doi/abs/10.1145/961268.961272 (visited on 01/02/2026).

[31] Zhenhua Liu et al. “Geographical load balancing with renewables”. In: *SIGMETRICS Perform. Eval. Rev.* 39.3 (Dec. 2011), pp. 62–66. ISSN: 0163-5999. DOI: 10.1145/2160803.2160862. URL: https://dl.acm.org/doi/10.1145/2160803.2160862 (visited on 09/16/2025).

[32] Zhenhua Liu et al. “Greening geographical load balancing”. In: *SIGMETRICS Perform. Eval. Rev.* 39.1 (June 2011), pp. 193–204. ISSN: 0163-5999. DOI: 10.1145/2007116.2007139. URL: https://dl.acm.org/doi/10.1145/2007116.2007139 (visited on 09/18/2025).

[33] Jens Malmodin, Nina Lövehagen, Pernilla Bergmark, and Dag Lundén. “ICT sector electricity consumption and greenhouse gas emissions – 2020 outcome”. In: *Telecommunications Policy* 48.3 (Apr. 2024), p. 102701. ISSN: 0308-5961. DOI: 10.1016/j.telpol.2023.102701. URL: https://www.sciencedirect.com/science/article/pii/S0308596123002124 (visited on 12/29/2025).

[34] Uttam Mandal et al. “Greening the cloud using renewable-energy-aware service migration”. In: *IEEE Network* 27.6 (Nov. 2013), pp. 36–43. ISSN: 1558-156X. DOI: 10.1109/MNET.2013.6678925. URL: https://ieeexplore.ieee.org/abstract/document/6678925 (visited on 08/07/2025).

[35] Pia Marchegiani, Elisa Morgera, and Louisa Parks. “Indigenous peoples’ rights to natural resources in Argentina: the challenges of impact assessment, consent and fair and equitable benefit-sharing in cases of lithium mining”. In: *The International Journal of Human Rights* 24 (Nov. 2019), pp. 1–17. DOI: 10.1080/13642987.2019.1677617.

[36] *ME SOLshare: Peer-to-Peer Smart Village Grids | Bangladesh | UNFCCC*. URL: https://unfccc.int/climate-action/momentum-for-change/ict-solutions/solshare (visited on 12/29/2025).

[37] Giovanni Neglia, Matteo Sereno, and Giuseppe Bianchi. “Geographical Load Balancing across Green Datacenters: A Mean Field Analysis”. In: *SIGMETRICS Perform. Eval. Rev.* 44.2 (Sept. 2016), pp. 64–69. ISSN: 0163-5999. DOI: 10.1145/3003977.3003998. URL: https://dl.acm.org/doi/10.1145/3003977.3003998 (visited on 09/18/2025).

[38] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. “An analysis of approximations for maximizing submodular set functions—I”. en. In: *Mathematical Programming* 14.1 (Dec. 1978), pp. 265–294. ISSN: 1436-4646. DOI: 10.1007/BF01588971. URL: https://doi.org/10.1007/BF01588971 (visited on 01/02/2026).

[39] *Off-grid Renewable Energy Statistics 2024*. URL: https://www.irena.org/Publications/2024/Dec/Off-grid-Renewable-Energy-Statistics-2024 (visited on 12/29/2025).

[40] James O’Donnellarchive page and Casey Crownhartarchive page. *We did the math on AI’s energy footprint. Here’s the story you haven’t heard.* en. URL: https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/ (visited on 12/29/2025).

[41] Sergey Paltsev. “The complicated geopolitics of renewable energy”. In: *Bulletin of the Atomic Scientists* 72.6 (Nov. 2016). Publisher: Routledge _eprint: https://doi.org/10.1080/00963402.2016.1240476, pp. 390–395. ISSN: 0096-3402. DOI: 10.1080/00963402.2016.1240476. URL: https://doi.org/10.1080/00963402.2016.1240476 (visited on 12/29/2025).

[42] Stefan Pfenninger and Iain Staffell. “Long-term patterns of European PV output using 30 years of validated hourly reanalysis and satellite data”. In: *Energy* 114 (Nov. 2016), pp. 1251–1265. ISSN: 0360-5442. DOI: 10.1016/j.energy.2016.08.060. URL: https://www.sciencedirect.com/science/article/pii/S0360544216311744 (visited on 01/02/2026).

[43] Jean-Marc Pierson et al. “DATAZERO: Datacenter With Zero Emission and Robust Management Using Renewable Energy”. In: *IEEE Access* 7 (2019), pp. 103209–103230. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2930368. URL: https://ieeexplore.ieee.org/abstract/document/8768369 (visited on 09/25/2025).

[44] Parakram Pyakurel. *Lithium is finite – but clean technology relies on such non-renewable resources*. en-EUROPE. Jan. 2019. DOI: 10.64628/AB.3y5da63dy. URL: http://theconversation.com/lithium-is-finite-but-clean-technology-relies-on-such-non-renewable-resources-109630 (visited on 01/08/2026).

[45] Tella Rajashekhar Reddy et al. *AI Greenferencing: Routing AI Inferencing to Green Modular Data Centers with Heron*. arXiv:2505.09989 [cs]. May 2025. DOI: 10.48550/arXiv.2505.09989. URL: http://arxiv.org/abs/2505.09989 (visited on 10/08/2025).

[46] *Renewable Energy & Environmental Justice - BOSCO Uganda*. en-US. Nov. 2025. URL: https://www.boscouganda.com/renewable-energy-environmental-justice/ (visited on 12/29/2025).

[47] *Renewables.ninja*. URL: https://www.renewables.ninja/ (visited on 01/02/2026).

[48] Bart Selman, David G. Mitchell, and Hector J. Levesque. “Generating hard satisfiability problems”. In: *Artificial Intelligence*. Frontiers in Problem Solving: Phase Transitions and Complexity 81.1 (Mar. 1996), pp. 17–29. ISSN: 0004-3702. DOI: 10.1016/0004-3702(95)00045-3. URL: https://www.sciencedirect.com/science/article/pii/0004370295000453 (visited on 01/02/2026).

[49] Himanshu Sharma, Ahteshamul Haque, and Zainul A. Jaffery. “Solar energy harvesting wireless sensor network nodes: A survey”. In: *Journal of Renewable and Sustainable Energy* 10.2 (Mar. 2018), p. 023704. ISSN: 1941-7012. DOI: 10.1063/1.5006619. URL: https://doi.org/10.1063/1.5006619 (visited on 08/06/2025).

[50] Petr Slavík. “A tight analysis of the greedy algorithm for set cover”. en. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing - STOC '96*. Philadelphia, Pennsylvania, United States: ACM Press, 1996, pp. 435–441. ISBN: 978-0-89791-785-8. DOI: 10.1145/237814.237991. URL: http://portal.acm.org/citation.cfm?doid=237814.237991 (visited on 01/02/2026).

[51] “Solar Protocol: Exploring Energy-Centered Design”. en. In: (2022).

[52] Thrasyvoulos Spyropoulos, Konstantinos Psounis, and Cauligi S. Raghavendra. “Spray and wait: an efficient routing scheme for intermittently connected mobile networks”. In: *Proceedings of the 2005 ACM SIGCOMM workshop on Delay-tolerant networking*. WDTN '05. New York, NY, USA: Association for Computing Machinery, Aug. 2005, pp. 252–259. ISBN: 978-1-59593-026-2. DOI: 10.1145/1080139.1080143. URL: https://dl.acm.org/doi/10.1145/1080139.1080143 (visited on 01/02/2026).

[53] Ion Stoica et al. “Chord: A scalable peer-to-peer lookup service for internet applications”. In: *SIGCOMM Comput. Commun. Rev.* 31.4 (Aug. 2001), pp. 149–160. ISSN: 0146-4833. DOI: 10.1145/964723.383071. URL: https://dl.acm.org/doi/10.1145/964723.383071 (visited on 09/15/2025).

[54] Konduru Sudharshan et al. “Systematic Review on Impact of Different Irradiance Forecasting Techniques for Solar Energy Prediction”. en. In: *Energies* 15.17 (Jan. 2022). Number: 17 Publisher: Multidisciplinary Digital Publishing Institute, p. 6267. ISSN: 1996-1073. DOI: 10.3390/en15176267. URL: https://www.mdpi.com/1996-1073/15/17/6267 (visited on 08/10/2025).

[55] Brian Sutherland. “Strategies for Degrowth Computing”. en. In: *Eighth Workshop on Computing within Limits 2022*. Virtual: LIMITS, June 2022. DOI: 10.21428/bf6fb269.04676652. URL: https://limits.pubpub.org/pub/strat (visited on 08/07/2025).

[56] Jennifer Switzer and Barath Raghavan. *Information Batteries: Storing Opportunity Power with Speculative Execution*. arXiv:2108.01035 [cs]. Aug. 2021. DOI: 10.48550/arXiv.2108.01035. URL: http://arxiv.org/abs/2108.01035 (visited on 09/09/2025).

[57] *The Rush For White Gold*. URL: https://www.earthisland.org/journal/index.php/magazine/entry/the-rush-for-white-gold/ (visited on 01/08/2026).

[58] Kannan Thirugnanam, Himanshu Saini, and Parvind Kumar. “Mathematical modeling of Li-ion battery for charge/discharge rate and capacity fading characteristics using genetic algorithm approach”. In: June 2012, pp. 1–6. ISBN: 978-1-4673-1407-7. DOI: 10.1109/ITEC.2012.6243431.

[59] *This is where your smartphone battery begins.* en. URL: https://www.washingtonpost.com/graphics/business/batteries/congo-cobalt-mining-for-lithium-ion-battery/ (visited on 01/08/2026).

[60] Thilo Wiertz, Lilith Kuhn, and Annika Mattissek. "A turn to geopolitics: Shifts in the German energy transition discourse in light of Russia's war against Ukraine". In: *Energy Research & Social Science* 98 (Apr. 2023), p. 103036. ISSN: 2214-6296. DOI: 10.1016/j.erss.2023.103036. URL: https://www.sciencedirect.com/science/article/pii/S2214629623000968 (visited on 12/29/2025).

[61] Nuzhat Yamin and Ganapati Bhat. "Online Solar Energy Prediction for Energy-Harvesting Internet of Things Devices". In: *2021 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. July 2021, pp. 1–6. DOI: 10.1109/ISLPED52811.2021.9502504. URL: https://ieeexplore.ieee.org/abstract/document/9502504 (visited on 08/10/2025).

[62] Haoyin Ye, Bo Yang, Yiming Han, and Nuo Chen. "State-Of-The-Art Solar Energy Forecasting Approaches: Critical Potentials and Challenges". English. In: *Frontiers in Energy Research* 10 (Mar. 2022). Publisher: Frontiers. ISSN: 2296-598X. DOI: 10.3389/fenrg.2022.875790. URL: https://www.frontiersin.org/journals/energy-research/articles/10.3389/fenrg.2022.875790/full (visited on 08/10/2025).

[63] Guanglin Zhang et al. "Distributed Energy Management for Multiple Data Centers With Renewable Resources and Energy Storages". In: *IEEE Transactions on Cloud Computing* 10.4 (Oct. 2022), pp. 2469–2480. ISSN: 2168-7161. DOI: 10.1109/TCC.2020.3031881. URL: https://ieeexplore.ieee.org/abstract/document/9234691 (visited on 10/01/2025).

[64] Daming Zhao and Jiantao Zhou. "An energy and carbon-aware algorithm for renewable energy usage maximization in distributed cloud data centers". In: *Journal of Parallel and Distributed Computing* 165 (July 2022), pp. 156–166. ISSN: 0743-7315. DOI: 10.1016/j.jpdc.2022.04.001. URL: https://www.sciencedirect.com/science/article/pii/S074373152200079X (visited on 08/07/2025).

Declaration

I acknowledge the use of generative AI (LLMs), in particular ChatGPT (`https://chat.openai.com/`), to help in tasks not essential to the contribution. In particular, it was used to help in writing `python` code to generate the figures, to help with LaTeX and TikZ content formatting, and to help for various debugging tasks.